TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Desarrollo de una aplicación basada en microservicios para el entrenamiento en auscultación médica.



Curso: 2025/2026

Alumno/a:

José Carlos Ruiz Sánchez

Director/es:

Antonio Jesús Fernandez García

A mi familia y amigos, por estar presente y ayudarme cuando lo necesitaba. Cada experiencia compartida, cada palabra y cada gesto han contribuido a formar quien soy a día de hoy. Gracias por ser parte de este viaje, con todos sus desafíos y alegrías.

ÍNDICE GENERAL

Página

Resumen y Abstract XIV

1. Introducción 1

1.1. Introducción e interés . . . 1
1.2. Motivación del trabajo de fin de grado . . . 1
1.3. Objetivos . . . 2
1.4. Planificación . . . 2

2. Background y Trabajos relacionados 7

2.1. Materiales y métodos . . . 7
2.2. Background . . . 13
2.2.1. Auscultación y tipos de sonidos patológicos . . . 13
2.2.2. Aprendizaje adaptativo, gamificación y comparación social . . . 14
2.2.3. Arquitectura de microservicios . . . 15
2.3. Trabajos relacionados . . . 16

3. Visión general 19

4. Especificación técnica y diseño 23

4.1. Definición de los requisitos . . . 23
4.1.1. Definición de los requisitos no funcionales . . . 23
4.1.2. Definición de los requisitos funcionales . . . 24
4.2. Documentación del modelado . . . 28
4.2.1. Diagrama de casos de uso . . . 29
4.2.2. Diagrama de base de datos . . . 29
4.3. Diseño conceptual . . . 32
4.4. Justificación de las tecnologías utilizadas. . . . 43
4.5. Implementación de las herramientas . . . 44
4.5.1. Preparación del Docker-compose . . . 44
4.5.2. Creación del proyecto Frontend . . . 46

4.5.3. Creación del proyecto Backend 46

5. Implementación **49**

5.1. Frontend 49

5.1.1. Inicio de sesión 49

5.1.2. Registrarse 50

5.1.3. Pestaña de inicio usuario normal 50

5.1.4. Estadísticas individuales 53

5.1.5. Siguiendo 54

5.1.6. Perfil 56

5.1.7. Responder preguntas 59

5.1.8. Perfil admin 62

5.1.9. Panel admin 62

5.1.10. Comparativa usuarios 63

5.2. Backend 64

5.2.1. Sincronizar datos. 64

5.2.2. Gestión autentificación. 64

5.2.3. Gestión perfil. 65

5.2.4. Gestión siguiendo. 65

5.2.5. Gestión categoría. 66

5.2.6. Gestionar preguntas. 66

5.2.7. Obtener algoritmos. 67

5.2.8. Gestor datos usuario preguntas. 67

6. Conclusión y trabajo futuro. **69**

6.1. Conclusiones 69

6.2. Trabajo futuro 70

BIBLIOGRAFÍA 70

ÍNDICE DE FIGURAS

1.1. Planificación inicial. 3
1.2. Planificación real del proyecto. 4

2.1. Logotipo de Figma. 7
2.2. Logotipo de Draw.io. 8
2.3. Logotipo de TypeScript. 8
2.4. Logotipo de Node.js. 9
2.5. Logotipo de Express.js. 9
2.6. Logotipo de Python. 9
2.7. Logotipo de Flask. 10
2.8. Logotipo de React. 10
2.9. Logotipo de Chart.js. 11
2.10. Logotipo de Visual Studio Code. 11
2.11. Logotipo de MySQL. 11
2.12. Logotipo de Vite. 12
2.13. Logotipo de Docker. 12
2.14. Logotipo de Microsoft Project. 12
2.15. Logotipo de Google Workspace. 13
2.16. Logotipo de Overleaf. 13

3.1. Diagrama general de la aplicación. 19
3.2. Diagrama de los microservicios de la aplicación. 20
3.3. Diagrama de arquitectura física de la aplicación. 21

4.1. Diagramas de casos de uso de la aplicación. 29
4.2. Versión preliminar de la base de datos. 30
4.3. Prototipo visual de registrarse. 33
4.4. Prototipo visual de iniciar sesión. 34
4.5. Prototipo visual de la pestaña de inicio. 35

4.6. Prototipo visual de la pestaña de inicio con desplegable. 35
4.7. Prototipo visual de la pestaña de preguntas. 36
4.8. Prototipo visual de la pestaña de preguntas cuando te equivocas. 37
4.9. Prototipo visual de la pestaña de preguntas cuando aciertas. 37
4.10. Prototipo visual de la pestaña de preguntas completadas. 38
4.11. Prototipo visual de la pestaña de estadísticas. 38
4.12. Prototipo visual de la pestaña de siguiendo sin seguidores. 39
4.13. Prototipo visual de la pestaña de siguiendo al buscar un usuario. 40
4.14. Prototipo visual de la pestaña de siguiendo al seguir al menos un usuario. . . . 40
4.15. Prototipo visual de la pestaña de estadísticas entre ambos usuarios. 41
4.16. Prototipo visual de la pestaña perfil. 41
4.17. Prototipo visual de la pestaña perfil al modificar el correo. 42
4.18. Prototipo visual de la pestaña perfil al modificar la contraseña. 42
4.19. Prototipo visual de la pestaña perfil al eliminar una cuenta. 43

5.1. Iniciar sesión final. 49
5.2. Registrarse final. 50
5.3. Inicio final. 51
5.4. Inicio con el desplegable abierto final. 51
5.5. Inicio con el desplegable abierto y con una descripción final. 52
5.6. Inicio con el desplegable de las categorías abierto. 52
5.7. Estadísticas de un usuario. 53
5.8. Estadísticas de un usuario donde se ve la información exacta. 54
5.9. Pestaña de siguiendo sin seguir a otro usuario. 54
5.10. Pestaña de siguiendo al buscar un usuario. 55
5.11. Pestaña de siguiendo al buscar un usuario que no existe. 55
5.12. Pestaña de siguiendo si se siguen usuarios. 56
5.13. Pestaña de siguiendo si se desea eliminar algún usuario. 56
5.14. Perfil final. 57
5.15. Perfil si se desea modificar el correo. 57
5.16. Perfil si se desea modificar la contraseña. 58
5.17. Perfil privado. 58
5.18. Mensaje de confirmación de eliminación de cuenta del perfil. 59
5.19. Pestaña para responder una pregunta. 59
5.20. Pestaña para responder una pregunta fallida. 60

5.21. Pestaña para responder una pregunta acertada. 60
5.22. Pestaña de resultados si has acertado menos de 5. 61
5.23. Pestaña de resultados si has acertado más de 5 y menos de 9. 61
5.24. Pestaña de resultados si has acertado todas las preguntas. 62
5.25. Perfil del admin. 62
5.26. Panel del admin. 63
5.27. Panel del admin. 63

ÍNDICE DE TABLAS

1.1. Tiempo dedicado por tarea. 5

4.1. RNF 1. Escalabilidad 23
4.2. RNF 2. Usabilidad 23
4.3. RNF 3. Seguridad 24
4.4. RF 1. Iniciar sesión 24
4.5. RF 2. Crear cuenta 24
4.6. RF 3. Elegir algoritmo 24
4.7. RF 4. Cerrar sesión 24
4.8. RF 5. Ver preguntas 25
4.9. RF 6. Reproducir audio de la pregunta 25
4.10. RF 7. Responder pregunta 25
4.11. RF 8. Saltar pregunta 25
4.12. RF 9. Actualizar estadísticas usuario 25
4.13. RF 10. Ver perfil 25
4.14. RF 11. Modificar correo 26
4.15. RF 12. Modificar contraseña 26
4.16. RF 13. Modificar estado público 26
4.17. RF 14. Ver perfil usuario registrado 26
4.18. RF 15. Eliminar cuenta 26
4.19. RF 16. Ver estadísticas 26
4.20. RF 17. Comparar estadísticas 27
4.21. RF 18. Buscar usuarios públicos 27
4.22. RF 19. Seguir usuario 27
4.23. RF 20. Actualizar lista de seguidores 27
4.24. RF 21. Ver lista siguiendo público 27
4.25. RF 22. Dejar de seguir 27
4.26. RF 23. Ver panel de administración 28

4.27. RF 24. Crear categoría 28
4.28. RF 25. Crear pregunta 28
4.29. RF 26. Eliminar pregunta 28
4.30. RF 27. Eliminar categoría 28
4.31. Entidad: Usuarios 31
4.32. Entidad: CriterioAlgoritmo 31
4.33. Entidad: UsuariosSeguidores 31
4.34. Entidad: UsuariosHasPreguntas 32
4.35. Entidad: Preguntas 32
4.36. Entidad: Categorias 32

LISTADOS

4.1. Dockerfile de frontend 44
4.2. Dockerfile de microservicio 45
4.3. Dockerfile de GestorAlgoritmos 45
4.4. Codigo de tsconfig.json 46

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

Este Trabajo de Fin de Grado tiene como propósito desarrollar una aplicación que ayude a estudiantes de medicina a reconocer sonidos corporales a partir de audios reales. Para ello, se ha diseñado una arquitectura basada en microservicios junto con un sistema de recomendación y elementos de gamificación.

El resultado es una aplicación funcional que genera sesiones de práctica personalizadas, permite gestionar preguntas y categorías, y ofrece estadísticas comparativas. Se concluye que la solución es escalable y constituye un recurso útil para apoyar el entrenamiento en auscultación.

The purpose of this Final Degree Project is to develop an application that helps medical students recognize body sounds using real audio samples. The system applies a microservices architecture together with a recommendation mechanism and gamification elements.

The final application is fully functional, generating personalized practice sessions, managing questions and categories, and providing comparative statistics. The results show that the solution is scalable and offers a useful tool to support auscultation training.

1 Introducción

1.1 Introducción e interés

En el ámbito de la educación médica se enfrenta el desafío constante de querer integrar vastos volúmenes de información teórica como habilidad práctica. Dentro de este contexto, el desarrollo de la capacidad de auscultación y reconocimiento de sonidos corporales, tanto normales como patológicos, emerge como un pilar fundamental en la formación futura de profesionales en el ámbito médico.

En el caso de la auscultación, esta requiere de una gran habilidad auditiva y destreza para asociar sonidos específicos a diagnósticos concretos. Sin embargo, la pericia en esta habilidad a menudo se ve bastante limitada por la disponibilidad de recursos y oportunidades de práctica en una clínica real.

Para ello, se ha realizado una aplicación cuyo objetivo consiste en enseñar al estudiante a reconocer audios para mejorar su destreza en este ámbito a partir de audios en los cuales debe elegir la respuesta correcta. También hace uso de un algoritmo para que elija las mejores preguntas en función de cómo avance el estudiante. Además de lo previamente citado, dicha aplicación irá añadiendo preguntas en función de los datos que introduzca el administrador de la aplicación para así disponer de una fuente de preguntas actualizada.

Además de lo previamente citado, se ha decidido utilizar un enfoque cercano y social en la aplicación para fomentar que el estudiante quiera aprender, buscando que la experiencia sea sencilla y agradable para así promover su motivación. Hay diversos estudios que demuestran que el enfoque utilizado para la comparativa entre usuarios conocidos mejora la eficiencia y la motivación para aprender, como se muestra en la bibliografía [1, 2, 3].

1.2 Motivación del trabajo de fin de grado

La motivación para realizar este Trabajo de Fin de Grado (TFG) radica en su potencial para transformar la manera en que los estudiantes de medicina adquieren y perfeccionan sus habilidades de auscultación. Al proporcionar un entorno de aprendizaje interactivo, personalizado y accesible, esta herramienta no solo facilita la memorización de sonidos, sino que también fomenta el desarrollo de un razonamiento clínico sólido, basado en la interpretación precisa de la información auditiva. Además, gracias a la implementación con una estructura de microservicios, permite la escalabilidad para futuras mejoras en la aplicación y la posibilidad de escalar con distintos lenguajes de programación utilizando los más adecuados para cada caso concreto.

Además de lo citado previamente, se han pedido opiniones a distintos profesionales de este ámbito y a estudiantes, y se ha llegado a la conclusión de que esta aplicación beneficiaría a la enseñanza debido a que actualmente se suele reservar para situaciones prácticas. Por lo tanto, un estudiante no tiene una manera cómoda de estudiar esta área por su cuenta.

1.3 Objetivos

En este proyecto se ha buscado desarrollar una aplicación funcional utilizando una estructura de microservicios para el backend y un frontend agradable para el usuario, que permita al estudiante aprender a reconocer sonidos corporales a partir de un algoritmo clásico de recomendación que devuelva un listado de preguntas para que se resuelvan. También permitirá al alumno conocer el progreso de las lecciones que ha ido realizando a partir de una serie de gráficas y paneles de información que podrá comparar con otros usuarios.

A rasgos generales, listando los puntos más importantes, este proyecto constará de las siguientes partes:

- Creación de la aplicación con una interfaz agradable y cómoda para el usuario.
- Desarrollo de gráficas y paneles para la visualización del progreso del usuario y la comparativa con otros estudiantes.
- Desarrollo de un algoritmo para la selección de las preguntas, que será implementado en el Complemento del Trabajo de Fin de Grado.
- Desarrollo de una estructura de microservicios para el backend para facilitar la escalabilidad de la aplicación.

El resto del informe está organizado de la siguiente manera: en el **Capítulo 1** se presentan la introducción, la motivación, los objetivos y la planificación realizada. El **Capítulo 2** recoge los materiales y métodos utilizados, así como el *background* necesario para contextualizar el proyecto, incluyendo apartados dedicados a la auscultación, al aprendizaje adaptativo y, de manera especialmente relevante, a la **arquitectura de microservicios**, pilar fundamental del desarrollo de la aplicación. En el **Capítulo 3** se ofrece una visión general de la estructura interna del sistema y de cómo se organizan los distintos servicios. El **Capítulo 4** detalla el estudio de la implementación, abarcando los requisitos, el modelado, el diseño conceptual, la justificación tecnológica y la preparación del entorno. El **Capítulo 5** describe el desarrollo completo del frontend y backend. Finalmente, el **Capítulo 6** presenta las conclusiones obtenidas y las posibles líneas de trabajo futuro.

1.4 Planificación

En este apartado se muestra de forma detallada la planificación seguida en la elaboración del presente Trabajo de Fin de Grado.

De acuerdo con la planificación inicial, el desarrollo del TFG se extendía desde el **29/12/2024** hasta el **31/05/2025**. Dentro de este intervalo se definieron varias fases principales del proyecto, cada una de ellas con un periodo teórico de ejecución asignado:

- **Planificación inicial**: del **29/12/2024** al **05/01/2025**. Esta fase se centró en la definición del alcance del proyecto, la identificación de tareas y la estimación de tiempos para el resto de fases.
- **Preparación del anteproyecto**: del **06/01/2025** al **10/01/2025**. Durante este intervalo se elaboró la propuesta inicial del Trabajo de Fin de Grado, incluyendo objetivos, metodología y planificación preliminar.
- **Formación con la tecnología utilizada**: del **13/01/2025** al **03/02/2025**. En esta fase se contempló la adquisición y consolidación de conocimientos necesarios sobre las herramientas y tecnologías empleadas en el desarrollo de la aplicación.
- **Desarrollo de la aplicación**: del **04/02/2025** al **29/04/2025**. Corresponde a la fase principal de implementación, en la que se diseñan, desarrollan y prueban las funcionalidades del sistema.
- **Redacción del documento**: del **13/01/2025** al **06/05/2025**. La redacción de la memoria se planificó de forma paralela al desarrollo técnico, con el fin de ir documentando progresivamente el trabajo realizado.
- **Elaboración y preparación de la presentación**: del **15/04/2025** al **15/05/2025**. Esta fase incluye la preparación de las diapositivas, el guion de la exposición y los ensayos de la defensa.

Además de estas fases principales, durante el desarrollo real del proyecto se llevaron a cabo otras tareas específicas que no figuraban como fases independientes en la planificación inicial, pero que han tenido un peso relevante en el trabajo realizado, como el **desarrollo de un prototipo visual**, la **creación y modelado de la base de datos** y el **desarrollo del diagrama de casos de uso y la definición de los requisitos funcionales**. Estas tareas se integran dentro de las fases de análisis y desarrollo, y quedan reflejadas de forma más detallada en la planificación real y en el reparto de horas.

A continuación se presentan dos diagramas de Gantt. En la figura 1.1 se recoge la planificación inicial del proyecto, basada en los intervalos teóricos asignados a cada fase, mientras que en la figura 1.2 se muestra la planificación real, donde se representan las fechas efectivas de ejecución de cada tarea y los reajustes temporales realizados a lo largo del desarrollo.



Figura 1.1: Planificación inicial.

Si se compara la figura 1.1 con la figura 1.2, se observa una desviación inicial considerable respecto al comienzo previsto del Trabajo de Fin de Grado. Esta divergencia principal

se debe a la hospitalización del redactor de este documento y a la imposibilidad de poder trabajar en el mismo durante aproximadamente un mes. Como consecuencia de este retraso, fue necesario reajustar la planificación para redistribuir las tareas y permitir que el proyecto pudiera abordarse dentro de los plazos establecidos, tal y como se aprecia en la figura 1.2.

En la planificación real, el Trabajo de Fin de Grado abarca un total de **287 días**, desde el **29/12/2024** hasta el **31/01/2026**. La fase de **planificación inicial** se mantiene entre el **29/12/2024** y el **05/01/2025** (7 días), seguida de la **preparación del anteproyecto**, que ocupa del **06/01/2025** al **10/01/2025** (5 días). La **formación con la tecnología utilizada** se desplaza en el tiempo y se desarrolla finalmente del **15/02/2025** al **03/03/2025** (12 días). El **desarrollo del prototipo visual (mockup)** tiene lugar entre el **15/03/2025** y el **19/03/2025** (4 días), mientras que la tarea de **casos de uso y requisitos funcionales** se extiende del **20/03/2025** al **03/04/2025** (11 días). A continuación, la **creación y modelado de la base de datos** se lleva a cabo entre el **04/04/2025** y el **12/04/2025** (7 días). La **redacción del documento** se prolonga de forma paralela al desarrollo técnico desde el **16/02/2025** hasta el **01/12/2025** (207 días), mientras que el **desarrollo del proyecto** se concentra entre el **30/04/2025** y el **15/10/2025** (121 días). Esta reorganización temporal refleja el reajuste necesario para absorber los retrasos iniciales e integrar de forma explícita las tareas de análisis y diseño que no figuraban de manera desglosada en la planificación teórica.

La comparación entre ambos diagramas permite analizar las diferencias entre la planificación teórica descrita anteriormente y la planificación real, tanto en términos de fechas de inicio y fin como de solapamiento y duración efectiva de cada fase. Además del desplazamiento temporal de las primeras fases, en la planificación real se observa una mayor granularidad en el trabajo, incorporando tareas específicas como el prototipo visual, el modelado de la base de datos y la definición detallada de requisitos, que no aparecían desglosadas en el Gantt inicial pero sí han supuesto un esfuerzo significativo en horas.

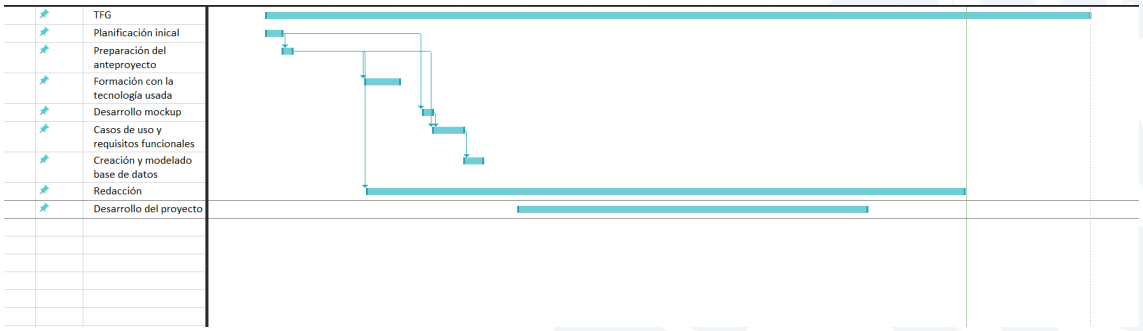


Figura 1.2: Planificación real del proyecto.

Para concluir, y con el objetivo de verificar el cumplimiento de las horas estimadas en el proyecto, se presenta la tabla 1.1, que detalla la distribución de horas por tarea y expone el tiempo dedicado a cada fase del desarrollo de este Trabajo de Fin de Grado. Con el fin de asegurar la precisión en la medición del tiempo, se ha empleado la técnica de Pomodoro, un método de gestión del tiempo que divide el trabajo en intervalos de 25 minutos separados por breves descansos.

En la tabla 1.1 se puede observar el tiempo real dedicado a cada una de las tareas registradas. En concreto, para la tarea de **planificación** se han dedicado **2 horas reales**,

mientras que la **preparación del anteproyecto** ha requerido **5 horas**. La **formación con la tecnología utilizada** ha supuesto **9 horas reales**, necesarias para adquirir los conocimientos previos al desarrollo.

Dentro de las tareas más directamente relacionadas con el desarrollo del sistema, el **desarrollo del prototipo visual** concentra **11 horas**, la **creación y modelado de la base de datos 3 horas** y el **desarrollo del diagrama de casos de uso y la definición de los requisitos funcionales 6 horas**. El **desarrollo de la aplicación** es la tarea con mayor carga de trabajo, con **60 horas reales**, al tratarse de la fase central de implementación. Por último, la **redacción del documento** ha requerido **47,30 horas**, mientras que la **elaboración y preparación de la presentación** ha supuesto **4 horas** adicionales.

Sumando todas las tareas, el proyecto acumula un total de **147,30 horas reales de dedicación**. Esta carga de trabajo se considera adecuada y coherente con la envergadura del Trabajo de Fin de Grado realizado: por un lado, permite abordar con profundidad las fases de análisis, diseño, desarrollo y validación de la aplicación; por otro, garantiza el tiempo necesario para documentar correctamente el proceso y preparar la defensa oral. El hecho de que, pese a los imprevistos iniciales, se haya podido completar el trabajo con una distribución equilibrada de horas entre las distintas fases indica que la planificación, aunque reajustada, ha sido realista y que se han cumplido los objetivos planteados tanto a nivel técnico como de documentación.

Tarea	**Horas (reales)**
Planificación	2,08
Preparación del anteproyecto	5,00
Formación con la tecnología utilizada	8,75
Desarrollo del prototipo visual	10,53
Creación y modelado de la base de datos	2,52
Desarrollo de diagrama de casos de uso y definición de requisitos funcionales	6,25
Desarrollo de la aplicación	60,00
Redacción del documento	47,30
Elaboración y preparación de la presentación	4,00

Tabla 1.1: Tiempo dedicado por tarea.

2 Background y Trabajos relacionados

2.1 Materiales y métodos

En esta sección se enumeran las herramientas, recursos y métodos utilizados durante el desarrollo del presente Trabajo de Fin de Grado.

Para la planificación inicial del proyecto se empleó **Microsoft Project**, con el fin de organizar las tareas y estimar la carga de trabajo prevista. Asimismo, se utilizó **Google Workspace** como apoyo para el registro de horas, la gestión de documentos y el seguimiento del progreso.

La definición de los requisitos funcionales y no funcionales, así como la elaboración de los casos de uso, se llevó a cabo con la herramienta **Draw.io**. Para el diseño visual de la interfaz se utilizó **Figma**, donde se crearon los prototipos que sirvieron como referencia para el posterior desarrollo.

En cuanto a las tecnologías empleadas, el frontend se implementó utilizando **React** junto con **Vite**, priorizando la eficiencia y la modularidad. Para el backend se seleccionaron **Node.js** y **Python**, valorando las ventajas que ofrecía cada lenguaje para las tareas correspondientes, especialmente en el desarrollo del algoritmo de recomendación.

Respecto al sistema de almacenamiento, se utilizó **MySQL** como gestor de bases de datos para soportar el esquema definido en el proyecto.

1. Diseño de la aplicación y modelado.

- **Figma**: Figma es una herramienta de diseño de interfaces de usuario que permite crear prototipos de forma sencilla y con funcionalidades para simular el comportamiento de la aplicación final. Fue desarrollada por Dylan Field y Evan Wallace, y se publicó en 2016 [4].



Figura 2.1: Logotipo de Figma.

- **Draw.io**: Draw.io, también conocido como diagrams.net, es una herramienta especializada en el diseño de diagramas. Fue creada por JGraph y permite generar diagramas de cualquier tipo sin necesidad de instalar software adicional, ya que funciona directamente en el navegador [5].



Figura 2.2: Logotipo de Draw.io.

2. Elaboración del código fuente.

Para el desarrollo del código fuente de la aplicación se han utilizado diversos lenguajes, librerías y herramientas con el objetivo de facilitar la construcción de cada uno de los componentes del sistema.

- **TypeScript**: TypeScript es un lenguaje de programación de código abierto desarrollado y mantenido por Microsoft. Es un superset de JavaScript, lo que implica que todo el código válido en JavaScript también lo es en TypeScript. Añade tipado estático y herramientas que facilitan el mantenimiento y la escalabilidad del software [6].



Figura 2.3: Logotipo de TypeScript.

- **Node.js**: Node.js es un entorno de ejecución de JavaScript basado en el motor V8 de Google Chrome. Permite construir aplicaciones de red escalables y de alto rendimiento. Es ampliamente utilizado en arquitecturas de microservicios y facilita la creación y gestión de APIs REST [7].



Figura 2.4: Logotipo de Node.js.

- **Express.js**: Express.js es un framework minimalista para Node.js que permite construir APIs y aplicaciones web de forma rápida y flexible. Proporciona herramientas sencillas para gestionar rutas, peticiones y middleware [8].



Figura 2.5: Logotipo de Express.js.

- **Python**: Python es un lenguaje de programación de alto nivel, interpretado y de propósito general. Destaca por su sintaxis clara y legible, lo que facilita su uso en proyectos educativos y científicos. Ofrece herramientas versátiles para la creación de algoritmos, como el sistema de recomendación implementado en este trabajo. Fue creado por Guido van Rossum y publicado por primera vez en 1991 [9].



Figura 2.6: Logotipo de Python.

- **Flask**: Flask es un microframework para Python orientado a la creación de aplicaciones web y APIs REST. Fue desarrollado por Armin Ronacher y se caracteriza por su ligereza y flexibilidad, permitiendo añadir únicamente los componentes necesarios [10].



Figura 2.7: Logotipo de Flask.

- **React.js**: React.js es una biblioteca de JavaScript diseñada para el desarrollo de interfaces de usuario de forma eficiente y modular. Se utiliza principalmente en aplicaciones web dinámicas. Fue desarrollada por Jordan Walke y actualmente es mantenida por Meta [11].

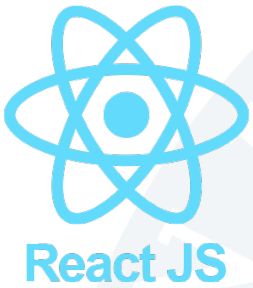


Figura 2.8: Logotipo de React.

- **Chart.js**: Chart.js es una biblioteca de JavaScript de código abierto diseñada para generar gráficos interactivos de manera sencilla. A partir de un conjunto de datos, permite visualizar información mediante diagramas dinámicos y personalizables [12].



Figura 2.9: Logotipo de Chart.js.

- **Visual Studio Code**: Visual Studio Code es un editor de código desarrollado por Microsoft. Destaca por su versatilidad y por su amplio conjunto de extensiones, que permiten trabajar con múltiples lenguajes y herramientas en un solo entorno [13].



Figura 2.10: Logotipo de Visual Studio Code.

3. Base de datos.

- **MySQL**: MySQL es un sistema de gestión de bases de datos relacional que permite almacenar, organizar y recuperar grandes cantidades de datos de manera eficiente. Fue creado originalmente por la empresa sueca MySQL AB y publicado en 1995. [14]



Figura 2.11: Logotipo de MySQL.

4. Despliegue.

- **Vite**: Vite es una herramienta de construcción y desarrollo frontend orientada a ofrecer un entorno rápido y eficiente para aplicaciones web modernas. Fue creada por Evan You, autor de Vue.js, y lanzada en 2020. En la actualidad es mantenida por VoidZero. [15]



Figura 2.12: Logotipo de Vite.

- **Docker**: Docker es una plataforma diseñada para desplegar componentes en contenedores, lo que permite aislar el código y gestionar sus dependencias para garantizar su correcta ejecución en cualquier entorno. Su uso facilita la portabilidad y evita conflictos entre distintas partes de la aplicación. [16]

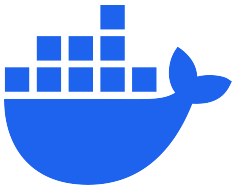


Figura 2.13: Logotipo de Docker.

5. Planificación y organización del tiempo.

- **Microsoft Project**: Microsoft Project es un software de gestión de proyectos diseñado para planificar, programar, asignar recursos, realizar seguimientos de progreso, administrar presupuestos y analizar cargas de trabajo. Fue desarrollado por Microsoft. [17]



Figura 2.14: Logotipo de Microsoft Project.

- **Google Workspace**: Google Workspace es un conjunto de aplicaciones que incluye procesador de texto, hojas de cálculo y herramientas de presentación. Permite gestionar documentos, colaborar en línea y organizar información de manera centralizada. Fue desarrollado por Google. [18]



Figura 2.15: Logotipo de Google Workspace.

6. Redacción.

- **Overleaf**: Overleaf es un entorno de escritura en línea diseñado específicamente para la creación de documentos mediante LaTeX. Fue desarrollado por WriteLaTeX Ltd. y permite trabajar de forma colaborativa, gestionar versiones y compilar documentos en la nube. [19]



Figura 2.16: Logotipo de Overleaf.

2.2 Background

Esta sección sirve para dar el contexto necesario respecto a lo que se ha buscado desarrollar en el presente TFG, ofreciendo distinta información necesaria para la absoluta comprensión.

Empezaremos primero con el ámbito de la medicina, seguido del ámbito del aprendizaje y acabando por último en el ámbito de la informática.

2.2.1 Auscultación y tipos de sonidos patológicos

La auscultación se define como la acción de escuchar los sonidos internos del cuerpo. Es una de las piedras angulares de la medicina. ¿Quién no ha ido al médico alguna vez y le han escuchado alguna parte del cuerpo a través de un estetoscopio? Esta técnica forma parte de una de las piedras angulares de la medicina; sin ella, muchas de las enfermedades quedarían sin diagnosticar y es, por tanto, vital entender y comprender en qué se basa [20].

El padre de esta técnica fue el médico René Laennec en el año 1816 [21]. A él nunca le gustó la técnica clásica de pegar la cabeza al paciente e intentar escuchar qué le ocurría; le parecía demasiado invasiva y poco precisa. Un día se le ocurrió la idea de utilizar un artilugio que se le ocurrió al ver niños jugar, que consistía en una especie de tubo de papel y pegar el oído. Este método resultó que servía para escuchar el cuerpo. A partir de aquí se fue perfeccionando hasta lo que conocemos hoy en día.

Lo malo de la auscultación es que es una habilidad muy compleja de desarrollar. Por un lado, se debe saber cómo colocar el estetoscopio, es decir, en qué parte del cuerpo colocarlo para escuchar lo que se necesite escuchar, y por otro lado se debe saber reconocer qué se está escuchando.

En el presente Trabajo de Fin de Grado nos centramos en cómo adquirir la habilidad de saber reconocer qué se está escuchando, porque no es algo fácil. Por un lado se debe saber reconocer el patrón fuera de lo común y, por otro lado, aprender a ignorar otros sonidos corporales que pueden dificultar el reconocimiento. La inclusión de tecnologías de simulación en educación médica ha demostrado incrementar la precisión y confianza en habilidades clínicas como la auscultación [22].

Esta es una de las áreas de la medicina más complejas de entrenar, porque normalmente requiere un paciente que sufra alguna condición concreta para poder practicar. Con los avances de las tecnologías, actualmente se pueden recoger audios patológicos; precisamente gracias a estos avances, es útil enseñar a partir de estos audios que ya se han recogido.

2.2.2 Aprendizaje adaptativo, gamificación y comparación social

Como se ha señalado en el subapartado anterior, la auscultación es un área muy importante de desarrollar. Debido a lo que se ha discutido, sirve para diagnosticar un sinfín de enfermedades. Esto nos hace preguntarnos: ¿cuál es la mejor forma de entrenar dicha habilidad?

Por un lado, se podría seguir utilizando la metodología clásica: esperar a que un paciente con ciertas condiciones patológicas aparezca y enseñar a los estudiantes con cada caso nuevo que ayude a enseñar. Pero este enfoque tiene un ligero problema. ¿No está demasiado influenciado por la suerte?, ¿no es demasiado poco eficiente? Y, en efecto, este método clásico no es el mejor.

El método clásico puede ofrecer cosas buenas; por ejemplo, son casos reales, es lo más parecido a lo que se va a encontrar un trabajador día tras día, porque suelen ser los casos más comunes, pero trae problemas con la consistencia de la enseñanza. Supongamos que un día hay un paciente con una patología demasiado concreta y rara. Como lo más probable es que se haya entrenado con unos datos del día a día, los más comunes, cuando suceda este caso raro lo más probable es que no se tenga la pericia para reconocerlo. Es por eso que se plantean otros enfoques para enseñar.

Un enfoque más moderno es utilizar los audios que se han podido recolectar gracias a los avances tecnológicos. Esto nos ahorra el problema de la consistencia, debido a que ahora sí disponemos tanto de las patologías raras como de las patologías comunes. Ante esto, nos hacemos la pregunta: ¿cuál es la mejor forma de enseñar ahora?

Por un lado, se puede seguir el enfoque más común en clases, que es la enseñanza por temario, es decir, primero se enseñan las patologías pulmonares, luego las cardíacas y así. Pero esto trae algunos problemas; por ejemplo, fomenta que el estudiante se olvide de lo que escuchó al inicio conforme va avanzando la asignatura o el tiempo. Además, no enseña de manera eficiente: a un alumno puede dársele bien el corazón, pero mal la zona abdominal.

En el presente TFG se plantea otro enfoque, un enfoque adaptativo, donde el estudiante elige qué estudiar y cuándo estudiarlo. También se propone que el sistema aprenda del

estudiante y de otros estudiantes para saber qué audios recomendar para afianzar así el conocimiento del estudiante. Estudios recientes muestran que los sistemas de aprendizaje adaptativo potenciados por inteligencia artificial pueden mejorar la personalización del aprendizaje [23].

Este enfoque trae el beneficio de que, si a un estudiante se le da mal algo, se reforzará en enseñar lo que se le da mal y, si algo se le olvida, se le refuerza. No sigue el enfoque clásico de temario al uso, porque cada estudiante tiene sus propias necesidades.

También el presente TFG tiene un enfoque relacionado con la gamificación. ¿Qué es la gamificación? La gamificación busca transmitir la enseñanza como un juego, algo para que el estudiante disfrute por aprender. La ludificación del aprendizaje como proceso para motivar el compromiso del estudiante ha sido descrita y aplicada en contextos educativos [24].

El enfoque de la gamificación es que, mediante un refuerzo positivo de dopamina por ir acertando y aprendiendo, el estudiante vaya deseando más y más aprender. Sigue un enfoque, por así decirlo, de refuerzo positivo, donde se premia por ir acertando.

El enfoque de la gamificación también requiere que sean sesiones cortas. El TFG se centra en darle 10 preguntas tipo test con 4 opciones; es una sesión en sí que no toma mucho rato en hacer. La idea subyace en que el estudiante no sienta que estudiar es un compromiso, sino que, si tiene un ratito libre, pueda repasar sin sentirse obligado a estar durante horas. Esto refuerza el deseo de aprender.

Además, por último estaría el enfoque de comparación social. Al centrar la aplicación en algo más sociable es cuando se inician ciertas competiciones entre estudiantes, entre amigos. Esto hace que puedan sentir el deseo por competir a ver quién tiene mejor promedio o quién consigue más respuestas acertadas en un día; es un enfoque que refuerza el aprendizaje a partir de un enfoque social [25].

2.2.3 Arquitectura de microservicios

Otro pilar fundamental del presente TFG es la estructura que se ha utilizado a la hora de desarrollarlo. Hay diversos enfoques, estructuras y patrones de diseño para desarrollar una aplicación; en nuestro caso se ha elegido una estructura de microservicios.

Normalmente, a la hora de desarrollar una aplicación, suele tener un enfoque monolítico, es decir, por un lado se programa el frontend en un lenguaje, por otro se programa el backend en otro lenguaje y se termina conectando de alguna forma. Este enfoque hace que, a la hora de desarrollar, se plantee bien qué lenguajes utilizar desde un inicio y suele ser un único lenguaje en cada parte.

También, en un enfoque más clásico de desarrollo, se suele crear un único directorio donde se recopilan los diversos archivos que mantienen dicha aplicación. Es cierto que cada archivo normalmente se dedica a hacer cosas independientes, pero aun así comparten espacio en el mismo directorio.

Estas prácticas anteriormente citadas pueden provocar diversos problemas durante el desarrollo y mantenimiento de la aplicación. Por un lado está el primer problema citado: elegir un único lenguaje para frontend, un único lenguaje para backend y detenernos aquí. Esto

puede provocar problemas por elegir algún lenguaje que no sea del todo óptimo para ciertas circunstancias.

Supongamos que tenemos una aplicación que requiere de mucha computación; estas partes de computación podría interesarnos programarlas con algún lenguaje de programación que consuma poco, por ejemplo C. Pero nos trae el problema de que otras partes, para las que sería más eficiente algún otro lenguaje, por ejemplo si necesitamos programar algún algoritmo además de la computación, sería más eficiente Python. Esto hace que sea muy difícil desarrollar estas partes, porque no es fácil unir distintos lenguajes en un mismo proyecto.

También está el problema de que suelen compartir los archivos un mismo directorio. Esto puede hacer que sea muy difícil escalar la aplicación o, si requiere alguna modificación, sea muy difícil encontrar qué se debe modificar, porque no se sepa bien qué hace cada archivo. Además, pueden existir problemas de dependencias.

Lo que se propone es algo más elegante y simple: un desarrollo basado en microservicios. ¿En qué se basa dicha estructura? La estructura de microservicios consiste en programar cada componente o servicio que haga algo como un componente independiente de los demás.

Supongamos que tenemos una tienda online. Una estructura de microservicios lo que nos dice es que, por un lado, programemos el servicio del carrito y compra, por otro programemos el sistema de login y registro, por otro el servicio de localización de pedidos, etc.

Básicamente, una estructura de microservicios busca granular una aplicación en componentes más pequeños para, posteriormente, juntarlos y formar una aplicación más grande.

Este enfoque soluciona los problemas citados anteriormente, debido a que cada microservicio, al ser independiente, puede programarse en el lenguaje que más se desee para la tarea que mejor requiera cada lenguaje. Como cada microservicio es independiente, no puede tener problemas de cohesión con el resto de códigos programados. Estudios recientes han analizado múltiples desafíos y prácticas en microservicios para apoyar el desarrollo sostenible de este tipo de sistemas [26].

También soluciona el problema de dependencias, ya que cada microservicio solo se encarga de lo que fue diseñado. Esto reduce problemas donde se modifica una parte de la aplicación y se rompe otra diferente, porque cada microservicio es independiente.

Por último, el mayor beneficio es que permite escalabilidad y modificaciones de manera eficiente. Se puede agregar un microservicio siempre que se quiera y, si se tiene que modificar una parte concreta, se acude al microservicio correspondiente y se hace el cambio.

2.3 TRABAJOS RELACIONADOS

En esta sección se detallarán otras aplicaciones similares que se utilizan actualmente en el mercado y que tienen un enfoque similar a la aplicación desarrollada en el presente Trabajo de Fin de Grado.

Empezaremos por 3M™ Littmann Learning Institute [27]. Es una de las aplicaciones más utilizadas para aprender la técnica de auscultación. Esto se debe a que ofrece una colección

completa de sonidos cardíacos, pulmonares y, en ocasiones, de otros tipos (como soplos carotídeos). Estos sonidos, a menudo grabados de pacientes reales, permiten al estudiante familiarizarse con la morfología acústica de patologías comunes y sonidos normales.

La mayor diferencia de Littmann Learning respecto a la aplicación desarrollada en el presente Trabajo de Fin de Grado es la metodología de enseñanza. En Littmann se utiliza una metodología más clásica y comúnmente empleada en lugares de enseñanza, frente a la metodología utilizada aquí, que está enfocada en la gamificación. Otra característica que la diferencia es el enfoque social: en el presente Trabajo de Fin de Grado se ha tenido muy en cuenta, mientras que en Littmann el enfoque es más solitario.

Otro trabajo relacionado es eMurmur [28]. Al igual que la aplicación desarrollada en el presente TFG, ambos abordan el mismo problema (la dificultad de aprender auscultación) desde dos perspectivas tecnológicas radicalmente distintas.

El enfoque de eMurmur se centra en la asistencia diagnóstica; su innovación principal es un motor de análisis (IA) que “escucha” por el usuario y le ofrece una interpretación. Su valor educativo deriva de exponer al estudiante a esta herramienta profesional. Enseña al estudiante en tiempo real con lo que escucha.

Esto puede acarrear problemas que el presente TFG soluciona. Lo primero es que se puede practicar sin necesidad de tener a un paciente ahí mismo; se puede entrenar a partir de lo subido en la aplicación y se pueden repasar los problemas que ya se han resuelto.

Otro trabajo relacionado muy parecido a eMurmur es Thinklabs iMurmur [29]. Al igual que el anterior, se enfoca en una enseñanza presencial. Como se discutió con eMurmur, el TFG se centra en un enfoque didáctico ajeno a que haya un paciente.

3 Visión general

En este capítulo se va a hablar de la estructura interna de la aplicación de este Trabajo de Fin de Grado. La aplicación sigue una estructura de microservicios, donde está dividida en cuatro directorios diferentes. A continuación se detallarán los directorios con más detalle.

Por un lado, está el directorio Frontend, donde se encuentra todo el código en React para todo el apartado visual de la aplicación.

También hay un directorio Backend, dentro del cual hay un directorio por cada servicio que ofrece la aplicación (GestionPreguntas, GestionAutentificación, GestorAlgoritmos).

También la aplicación cuenta con un directorio para la base de datos, donde se encuentra el archivo SQL para la inicialización del esquema, y, por último, un directorio donde se almacenan los audios de las preguntas. Véase la siguiente figura 3.1, donde se detalla mejor visualmente.

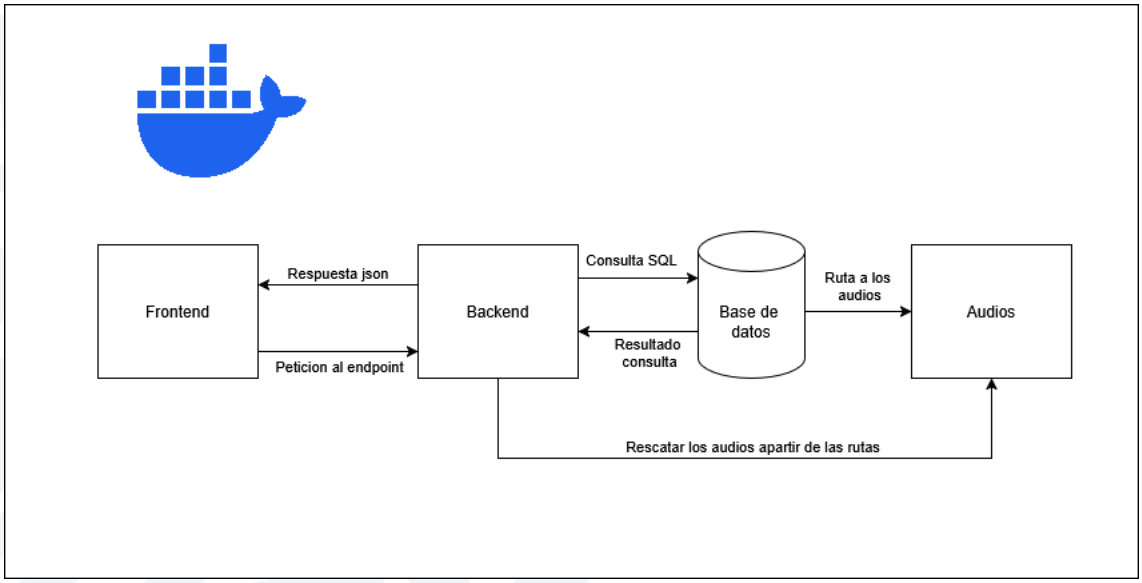


Figura 3.1: Diagrama general de la aplicación.

Si ampliamos cómo se relaciona el Backend respecto al Frontend, podemos ver una imagen más precisa de cómo los microservicios interactúan con la parte visual. Como se puede ver en la figura 3.2, cada microservicio es independiente, no hay relación entre ellos, todos se conectan al núcleo central, que es el Frontend, y todos llaman a la base de datos. Para simplificar el entendimiento, se ha decidido no incluir la base de datos en el siguiente diagrama.

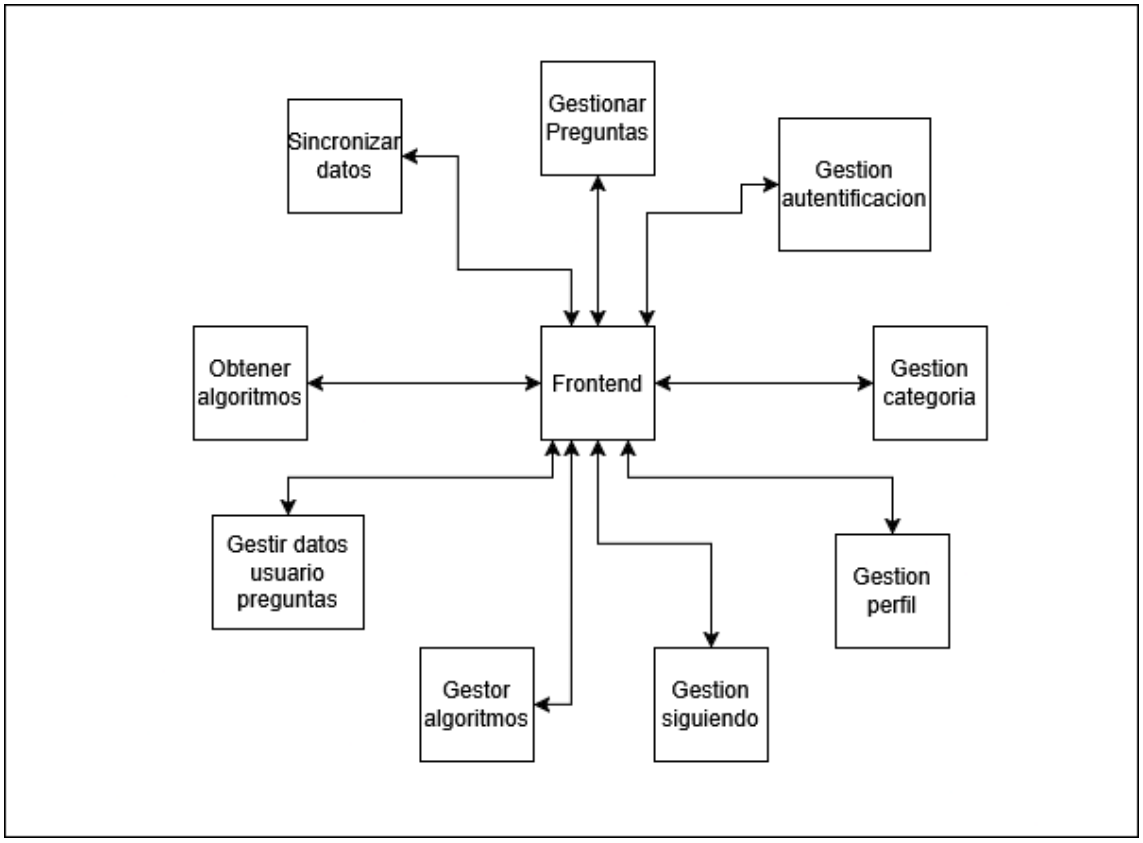


Figura 3.2: Diagrama de los microservicios de la aplicación.

Los beneficios de seguir esta estructura frente a un desarrollo tradicional consisten en que permite añadir fácilmente funcionalidades adicionales al proyecto. También permite utilizar distintos lenguajes de programación en función de lo que se necesite en ese momento. Permite también que sea más fácil modificar un servicio de la aplicación.

En el caso del presente Trabajo de Fin de Grado, esta estructura ha permitido la utilización de distintos lenguajes en el backend. La mayoría de microservicios están en Node.js, pero, para los algoritmos, se ha podido utilizar Python. También, si se quisiera en el día de mañana ampliar la aplicación y añadir funcionalidades, ofrece la posibilidad de ampliarla creando un nuevo directorio que represente un microservicio nuevo.

Esta estructura se ha logrado gracias a la utilización de Docker y Dockerfiles. Las razones por las que se utilizó esta herramienta y no Kubernetes se deben a que se busca minimizar la complejidad operativa, mejorar la eficiencia de recursos y aumentar la rapidez del despliegue.

Con el objetivo de complementar los diagramas anteriores, que describen principalmente la estructura lógica y funcional del sistema, se incluye a continuación un diagrama de arquitectura física o de red. Este tipo de diagrama permite visualizar cómo se despliegan los microservicios en la infraestructura y cómo se comunican entre sí y con los elementos externos del sistema.

En la figura 3.3 se muestra la arquitectura física de la aplicación. El Frontend, desarrollado en React, actúa como cliente y se comunica con el Backend a través de peticiones HTTP. Cada microservicio se despliega de manera independiente dentro de un contenedor

Docker, manteniendo así el principio de aislamiento propio de una arquitectura basada en microservicios.

Cada microservicio expone su propia API REST, a través de la cual recibe las peticiones necesarias para llevar a cabo su funcionalidad. Aunque conceptualmente cada API es independiente, en el diagrama se representan los microservicios como unidades autónomas que gestionan internamente sus endpoints. Todos los microservicios se conectan a una base de datos común, si bien cada uno accede únicamente a las tablas correspondientes a su dominio funcional.

El uso de Docker permite encapsular cada microservicio junto con sus dependencias, facilitando el despliegue, la escalabilidad y el mantenimiento del sistema. Esta arquitectura física refleja cómo la aplicación se ejecuta en un entorno real, complementando así la visión lógica presentada anteriormente.

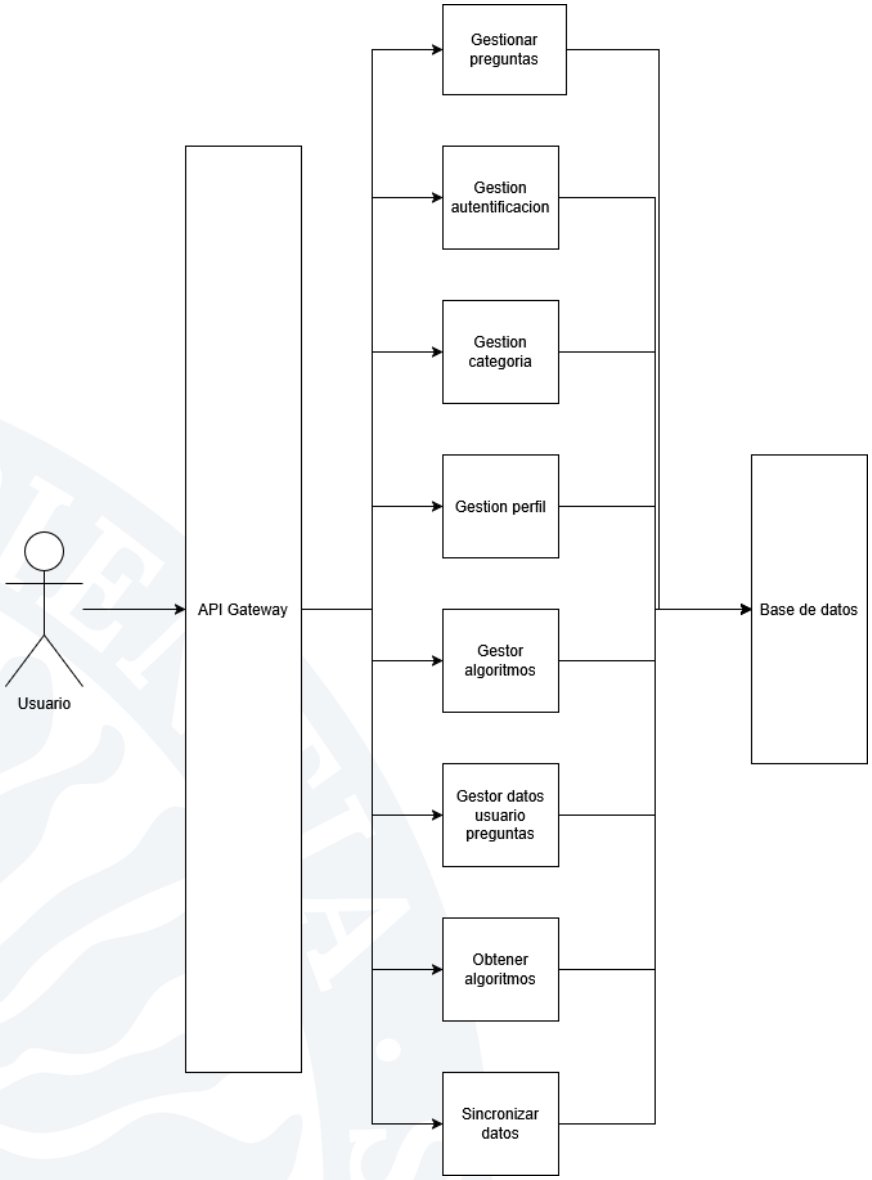


Figura 3.3: Diagrama de arquitectura física de la aplicación.

4 Especificación técnica y diseño

Este capítulo entra en detalle en lo desarrollado en la aplicación, mostrando la planificación inicial, la definición del problema en requisitos y casos de uso, además del diseño visual de la aplicación, tanto el diseño preliminar hecho en Figma como el diseño final de la aplicación.

4.1 Definición de los requisitos

En este apartado se detalla la definición de los requisitos tanto funcionales como no funcionales que se han planteado a lo largo del desarrollo del presente Trabajo de Fin de Grado.

4.1.1 Definición de los requisitos no funcionales

Los requisitos no funcionales son todas las características que debe seguir la aplicación, enfocados en cómo debe funcionar y en el sentido de qué tiene que hacer. Por ejemplo, supongamos que quisiéramos crear una casa: los requisitos funcionales serían tener un dormitorio, un baño, una cocina, etc. mientras que un requisito no funcional sería tener un aislante térmico, tener seguridad, etc.

Teniendo en cuenta lo citado anteriormente, se han definido los siguientes requisitos no funcionales:

RNF-01	Escalabilidad
Tipo de requisito	No funcional
Descripción	La aplicación será escalable para poder introducir funciones y mejoras nuevas.

Tabla 4.1: RNF 1. Escalabilidad

RNF-02	Usabilidad
Tipo de requisito	No funcional
Descripción	La aplicación tendrá como objetivo ser agradable para los usuarios y fácil de usar.

Tabla 4.2: RNF 2. Usabilidad

RNF-03	Seguridad
Tipo de requisito	No funcional
Descripción	La aplicación garantizará la seguridad de las credenciales de los usuarios mediante el cifrado de contraseñas utilizando el algoritmo bcrypt, evitando su almacenamiento en texto plano.

Tabla 4.3: RNF 3. Seguridad

4.1.2 Definición de los requisitos funcionales

Los requisitos funcionales son todas las características que debe tener la aplicación, enfocadas a los métodos y servicios que realiza la aplicación.

En el caso del presente Trabajo de Fin de Grado se han definido los siguientes requisitos funcionales:

RF-01	Iniciar sesión
Tipo de requisito	Funcional
Descripción	En la aplicación se podrá iniciar sesión.

Tabla 4.4: RF 1. Iniciar sesión

RF-02	Crear cuenta
Tipo de requisito	Funcional
Descripción	En la aplicación se podrá crear una cuenta.

Tabla 4.5: RF 2. Crear cuenta

RF-03	Elegir algoritmo
Tipo de requisito	Funcional
Descripción	Un usuario podrá solicitar preguntas a partir de elegir un algoritmo de los disponibles en la aplicación.

Tabla 4.6: RF 3. Elegir algoritmo

RF-04	Cerrar sesión
Tipo de requisito	Funcional
Descripción	Cuando un usuario haya iniciado sesión, podrá cerrar su sesión en la aplicación.

Tabla 4.7: RF 4. Cerrar sesión

RF-05	Ver preguntas
Tipo de requisito	Funcional
Descripción	Cuando un usuario haya solicitado una serie de preguntas, podrá visualizarlas para posteriormente responderlas, saltar la pregunta o reproducir el audio asociado.

Tabla 4.8: RF 5. Ver preguntas

RF-06	Reproducir audio de la pregunta
Tipo de requisito	Funcional
Descripción	Un usuario podrá escuchar el audio asociado a una pregunta.

Tabla 4.9: RF 6. Reproducir audio de la pregunta

RF-07	Responder pregunta
Tipo de requisito	Funcional
Descripción	Un usuario podrá contestar una pregunta.

Tabla 4.10: RF 7. Responder pregunta

RF-08	Saltar pregunta
Tipo de requisito	Funcional
Descripción	Un usuario podrá saltar una pregunta.

Tabla 4.11: RF 8. Saltar pregunta

RF-09	Actualizar estadísticas usuario
Tipo de requisito	Funcional
Descripción	Cuando un usuario responda las preguntas, se actualizarán los datos del usuario.

Tabla 4.12: RF 9. Actualizar estadísticas usuario

RF-10	Ver perfil
Tipo de requisito	Funcional
Descripción	Un usuario podrá ver su perfil para modificar posteriormente datos del propio usuario.

Tabla 4.13: RF 10. Ver perfil

RF-11	Modificar correo
Tipo de requisito	Funcional
Descripción	Un usuario podrá modificar el correo que utiliza en la aplicación, siempre y cuando no lo esté usando otro usuario.

Tabla 4.14: RF 11. Modificar correo

RF-12	Modificar contraseña
Tipo de requisito	Funcional
Descripción	Un usuario podrá modificar la contraseña que utiliza en la aplicación.

Tabla 4.15: RF 12. Modificar contraseña

RF-13	Modificar estado público
Tipo de requisito	Funcional
Descripción	Un usuario podrá modificar si su perfil es público o privado.

Tabla 4.16: RF 13. Modificar estado público

RF-14	Ver perfil usuario registrado
Tipo de requisito	Funcional
Descripción	Un usuario registrado tiene una función extra para eliminar la cuenta.

Tabla 4.17: RF 14. Ver perfil usuario registrado

RF-15	Eliminar cuenta
Tipo de requisito	Funcional
Descripción	Un usuario registrado puede eliminar su cuenta.

Tabla 4.18: RF 15. Eliminar cuenta

RF-16	Ver estadísticas
Tipo de requisito	Funcional
Descripción	Un usuario puede ver sus estadísticas en la aplicación.

Tabla 4.19: RF 16. Ver estadísticas

RF-17	Comparar estadísticas
Tipo de requisito	Funcional
Descripción	Un usuario puede comparar sus estadísticas con las estadísticas de otro usuario público al que siga.

Tabla 4.20: RF 17. Comparar estadísticas

RF-18	Buscar usuarios públicos
Tipo de requisito	Funcional
Descripción	Un usuario puede buscar a otros usuarios públicos que estén registrados en la aplicación.

Tabla 4.21: RF 18. Buscar usuarios públicos

RF-19	Seguir usuario
Tipo de requisito	Funcional
Descripción	Un usuario puede seguir a un usuario al que haya buscado en el buscador.

Tabla 4.22: RF 19. Seguir usuario

RF-20	Actualizar lista de seguidores
Tipo de requisito	Funcional
Descripción	Cuando se realiza alguna modificación de la lista de seguidores, esta se actualiza visualmente con el cambio.

Tabla 4.23: RF 20. Actualizar lista de seguidores

RF-21	Ver lista siguiendo público
Tipo de requisito	Funcional
Descripción	Se muestra una lista de los usuarios públicos que sigues.

Tabla 4.24: RF 21. Ver lista siguiendo público

RF-22	Dejar de seguir
Tipo de requisito	Funcional
Descripción	Un usuario puede dejar de seguir a otro usuario.

Tabla 4.25: RF 22. Dejar de seguir

RF-23	Ver panel de administración
Tipo de requisito	Funcional
Descripción	Un administrador puede acceder al panel de configuración de la aplicación.

Tabla 4.26: RF 23. Ver panel de administración

RF-24	Crear categoría
Tipo de requisito	Funcional
Descripción	Un administrador puede crear una categoría nueva en la aplicación.

Tabla 4.27: RF 24. Crear categoría

RF-25	Crear pregunta
Tipo de requisito	Funcional
Descripción	Un administrador puede crear una pregunta nueva en la aplicación.

Tabla 4.28: RF 25. Crear pregunta

RF-26	Eliminar pregunta
Tipo de requisito	Funcional
Descripción	Un administrador puede eliminar una pregunta de la aplicación.

Tabla 4.29: RF 26. Eliminar pregunta

RF-27	Eliminar categoría
Tipo de requisito	Funcional
Descripción	Un administrador puede eliminar una categoría de la aplicación con las preguntas asociadas a la categoría.

Tabla 4.30: RF 27. Eliminar categoría

4.2 Documentación del modelado

En esta sección se detallan todos los elementos visuales que han facilitado el desarrollo de la aplicación en la fase de implementación, este apartado recoge todos los diagramas, imágenes conceptuales de la aplicación e información relevante de la base de datos. Se ofrecerán detalles también de cómo ha ido variando la aplicación desde las fases más conceptuales hasta las más definitivas.

4.2.1 Diagrama de casos de uso

En este apartado se muestra el diagrama de casos de uso de la aplicación para facilitar el entendimiento de cómo funciona la aplicación. El diagrama juega un papel crucial en la definición clara de las funcionalidades del sistema. A continuación se puede ver en la siguiente figura 4.1 el diagrama de casos de uso del presente Trabajo de Fin de Grado.

Como se puede ver en la figura 4.1, en el diagrama se encuentran presentes 4 actores que participan en la aplicación. El actor UsuarioSinRegistrar solo puede iniciar sesión y crear cuenta y el actor Usuario es un actor genérico que sirve para juntar las funcionalidades del UsuarioRegistrado y el admin.

Si nos fijamos, el Usuario y el UsuarioRegistrado hacen prácticamente lo mismo, la mayor diferencia es que el UsuarioRegistrado puede eliminar la cuenta, por eso se ha separado el actor, dado que el admin no puede eliminar la cuenta en el perfil.

El admin está separado debido a que tiene una serie de funcionalidades únicas que mantienen la aplicación funcionando.

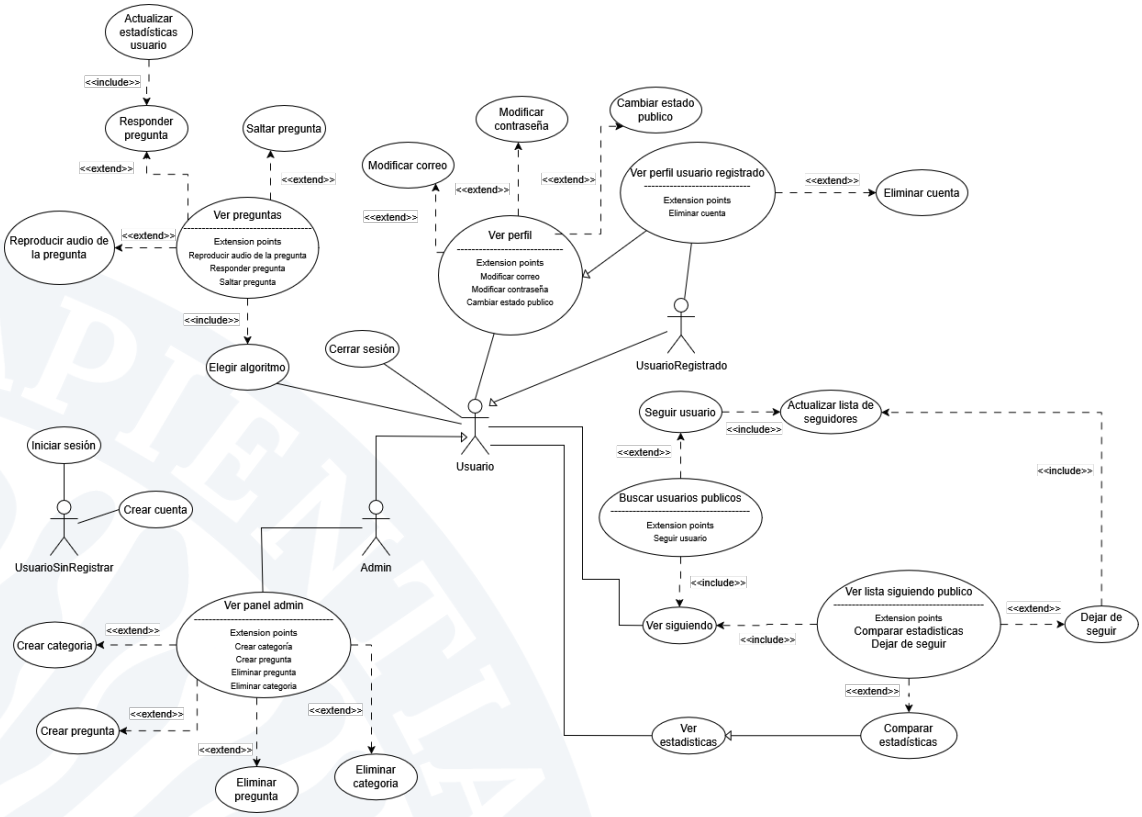


Figura 4.1: Diagramas de casos de uso de la aplicación.

4.2.2 Diagrama de base de datos

En este apartado se muestra el diagrama de base de datos usado en la aplicación, también se detallan las tablas de la base de datos, además de explicar la relación entre las tablas y el propósito de las mismas.

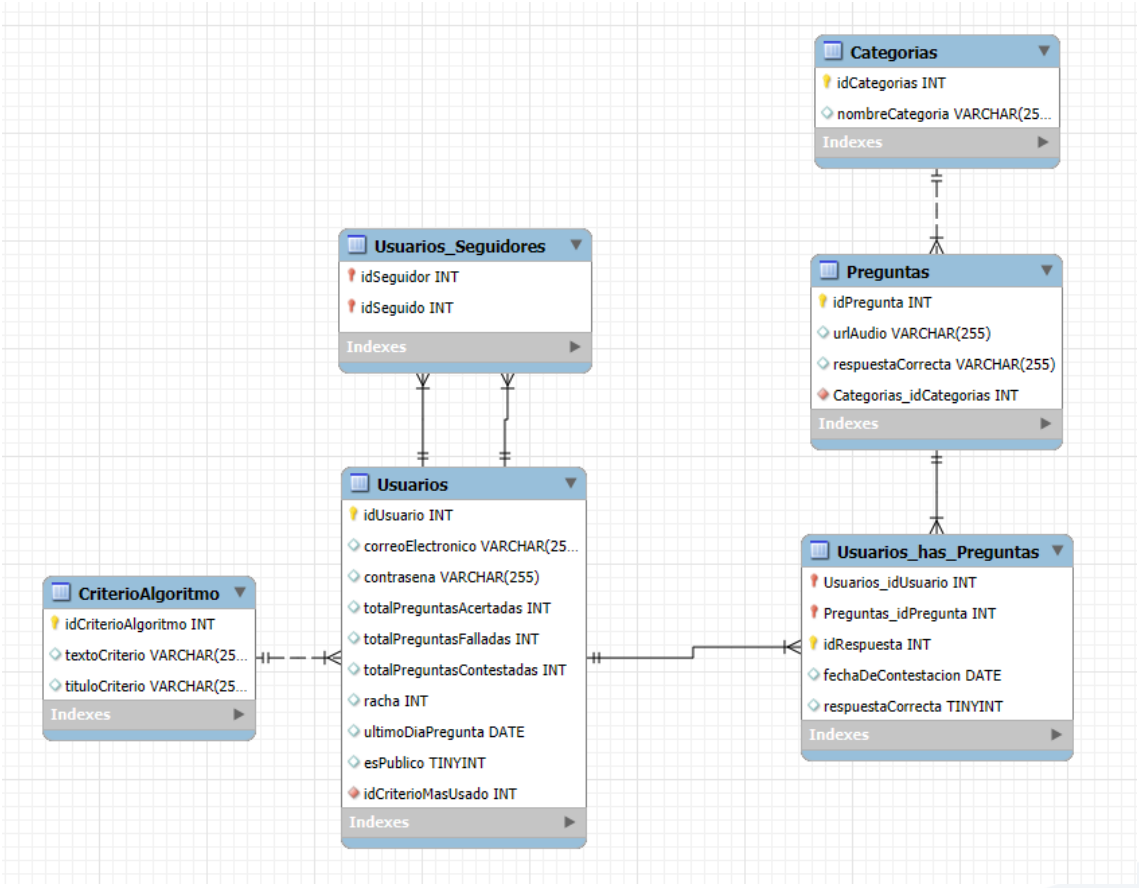


Figura 4.2: Versión preliminar de la base de datos.

A continuación se ven las siguientes tablas en mayor detalle.

La entidad usuarios, que se puede ver en la tabla 4.31, se encarga de guardar diversa información de interés de los usuarios que participan en el sistema, se guarda un id para el usuario que es auto incremental, junto a información de login. La contraseña se encripta a la hora de guardarla para aumentar la seguridad de la aplicación. El campo ultimoDiaPregunta se utiliza principalmente para saber cuándo fue la última vez que un usuario respondió una pregunta y se comprueba con la fecha actual para aumentar la racha, o resetearla a cero en caso de que haya habido algún día que no se haya respondido ninguna pregunta.

Entidad	Usuarios	
PK	idUsuario	INT
	correoElectronico	VARCHAR(255)
	contrasena	VARCHAR(255)
	totalPreguntasAcertadas	INT
	totalPreguntasFalladas	INT
	totalPreguntasContestadas	INT
	racha	INT
	ultimoDiaPregunta	DATE
	esPublico	TINYINT
FK	idCriterioMasUsado	INT

Tabla 4.31: Entidad: Usuarios

La entidad CriterioAlgoritmo que se puede ver en la tabla 4.32 se encarga de almacenar toda la información referente a los algoritmos de recomendación de la aplicación. En el presente documento no se detallará demasiado la importancia de esta tabla debido a que dicha relevancia se encuentra en el Complemento de Trabajo de Fin de Grado.

Entidad	CriterioAlgoritmo	
PK	idCriterioAlgoritmo	INT
	textoCriterio	VARCHAR(255)
	tituloCriterio	VARCHAR(255)

Tabla 4.32: Entidad: CriterioAlgoritmo

La entidad UsuariosSeguidores que se puede ver en la tabla 4.33 se encarga de almacenar la información necesaria referente a cuando un usuario sigue a otro en el sistema.

Entidad	UsuariosSeguidores	
PK	idSeguidor	INT
PK	idSeguido	INT

Tabla 4.33: Entidad: UsuariosSeguidores

La entidad UsuariosHasPreguntas que se puede ver en la tabla 4.34 se encarga de almacenar la relación entre un usuario y las preguntas que ha respondido, un usuario puede responder varias veces a la misma pregunta, con lo que para no perder el histórico de veces que se ha respondido una pregunta, se ha decidido poner una clave adicional a la tabla que es idRespuesta. También se ha tomado la decisión de tener un booleano que, si es true, simboliza que la respuesta ha sido contestada correctamente y si es un false, simboliza que ha sido contestado incorrectamente.

Entidad	UsuariosHasPreguntas	
FK	Usuarios idUsuario	INT
FK	Preguntas idPregunta	INT
PK	idRespuesta	INT
	fechaDeContestacion	DATE
	respuestaCorrecta	TINYINT

Tabla 4.34: Entidad: UsuariosHasPreguntas

La entidad Preguntas que se puede ver en la tabla 4.35 se encarga de almacenar toda la información relacionada con las preguntas, esta tabla sirve para almacenar la categoría de la pregunta, con la url de donde está y la respuesta correcta. En la aplicación, para las otras opciones incorrectas se elegirán preguntas de la misma categoría aleatoriamente.

Entidad	Preguntas	
PK	idPregunta	INT
	urlAudio	VARCHAR(255)
	respuestaCorrecta	VARCHAR(255)
FK	Categorias idCategorias	INT

Tabla 4.35: Entidad: Preguntas

La entidad Categorias que se puede ver en la tabla 4.36 se encarga de almacenar toda la información relacionada con las categorías, esta tabla sirve para almacenar a qué sección pertenece cada pregunta, para así poder filtrar de mejor forma qué enseñar a los estudiantes.

Entidad	Categorias	
PK	idCategorias	INT
	nombreCategoria	VARCHAR(255)

Tabla 4.36: Entidad: Categorias

4.3 Diseño conceptual

En este apartado se detallan todos los mockups y prototipos visuales que sirvieron como base para el desarrollo de la aplicación. Estos prototipos visuales fueron diseñados con Figma.

Estos prototipos visuales tienen algunas carencias en el desarrollo o ideas muy tempranas en el proyecto que se cambiaron con la elaboración del presente Trabajo de Fin de Grado.

A continuación se muestran las imágenes junto a una explicación de la idea original del diseño, qué cambios ha recibido cada imagen, a su vez, de una argumentación de por qué se ha decidido dicho cambio.

Como se puede ver en la figura 4.3, es un diseño básico de una creación de cuenta, aquí se introducía un correo electrónico y la contraseña. Los cambios que se han desarrollado son que ahora se muestra un mensaje de error en caso de que el correo electrónico no sea un correo electrónico que pueda existir, en caso de que un correo electrónico ya lo esté usando

un usuario o si las contraseñas introducidas no coinciden, también si no se han introducido todos los datos. Además, pulsar "¿No tienes cuenta? ¡Inicia sesión!" te redirige a la figura 4.4 y pulsar crear cuenta te redirige a la figura 4.5.

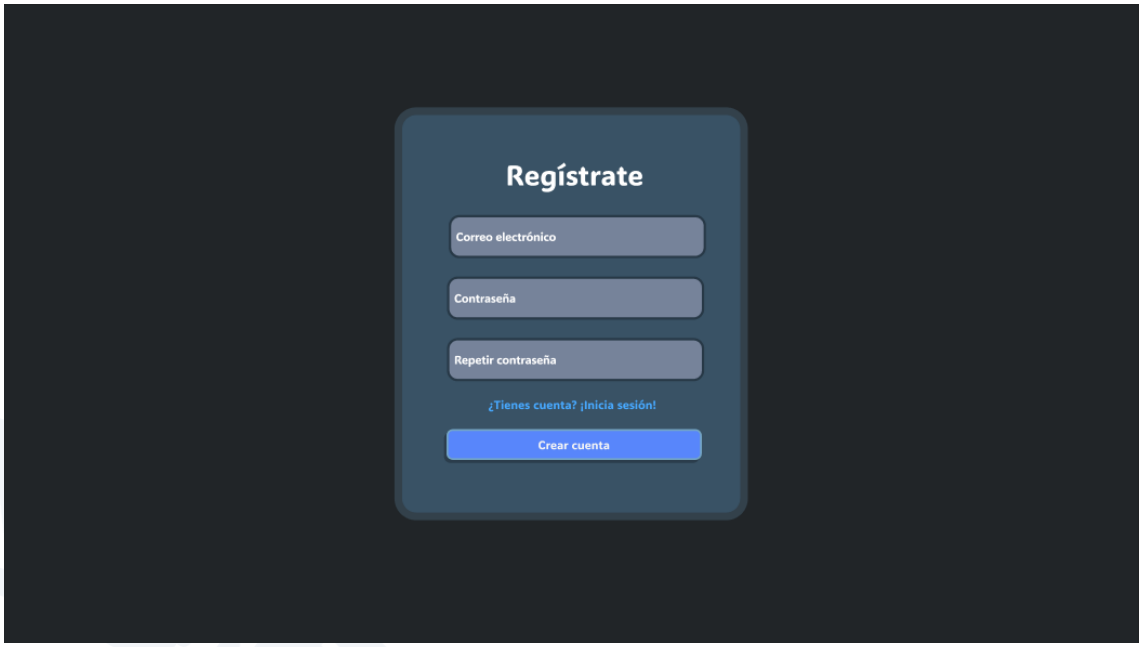


Figura 4.3: Prototipo visual de registrarse.

Como se puede ver en la figura 4.4, es un diseño básico de inicio de sesión, aquí se introducía un correo electrónico y la contraseña. Los cambios que se han desarrollado son que ahora se muestra un mensaje de error en caso de que no se hayan introducido todos los datos y si la contraseña no coincide con el correo electrónico. Además, pulsar "¿No tienes cuenta? ¡Regístrate con nosotros!" te redirige a la figura 4.3 y pulsar el botón de iniciar sesión te redirige a la figura 4.5.

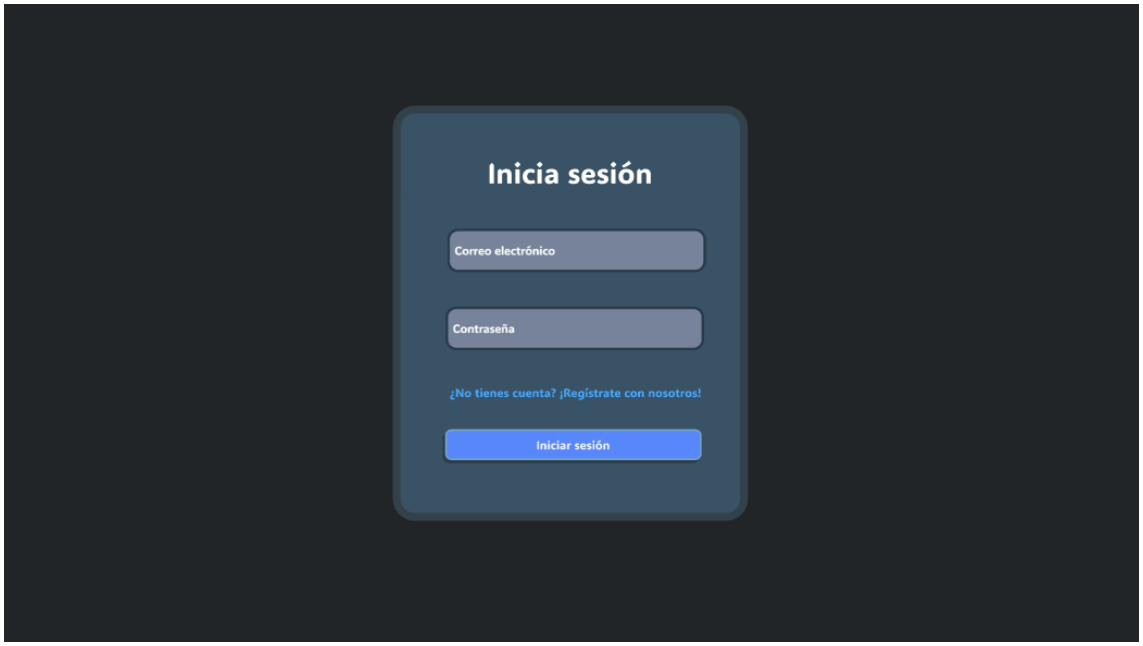


Figura 4.4: Prototipo visual de iniciar sesión.

Como se puede ver en la figura 4.5, es un diseño del prototipo original de la pestaña de inicio. En esta pestaña es el núcleo central donde se basa la aplicación y también es la que ha sufrido más cambios.

Uno de los cambios consiste en que ahora no hay un cuadro de texto central, esto era debido a que originalmente se planeaba utilizar un agente de IA para que seleccionara tus series de preguntas en función del criterio, al final se optó por un algoritmo clásico de recomendación donde está más explicado en el complemento.

En el lado de la izquierda se encuentra un menú que se comparte en distintas vistas de la aplicación, mirando de arriba abajo está el logo provisional, que al pulsarlo te redirige a la figura 4.5; del mismo modo, el botón aprender también te redirige ahí. El botón estadísticas te redirige a la figura 4.11, el botón siguiendo te lleva a la figura 4.12 o también te puede llevar a la figura 4.14 en función de si sigues a alguien o no, el botón perfil te lleva a la figura 4.16 y, por último, el botón cerrar sesión te redirige a la figura 4.4.

Por último, solo queda por explicar la parte central, en esta parte central si se pulsaba las 3 rayas, te salía un menú desplegable como se puede ver en la figura 4.6 y, al pulsar la búsqueda o alguna opción, te llevaba a la 4.7.



Figura 4.5: Prototipo visual de la pestaña de inicio.

Como se puede ver en la figura 4.6, hay distintas opciones que toman en cuenta el criterio de la preferencia del usuario e información del usuario; se ha tomado la decisión de que estas opciones se muestren en todo momento y sea así como interactúe el usuario con la aplicación.

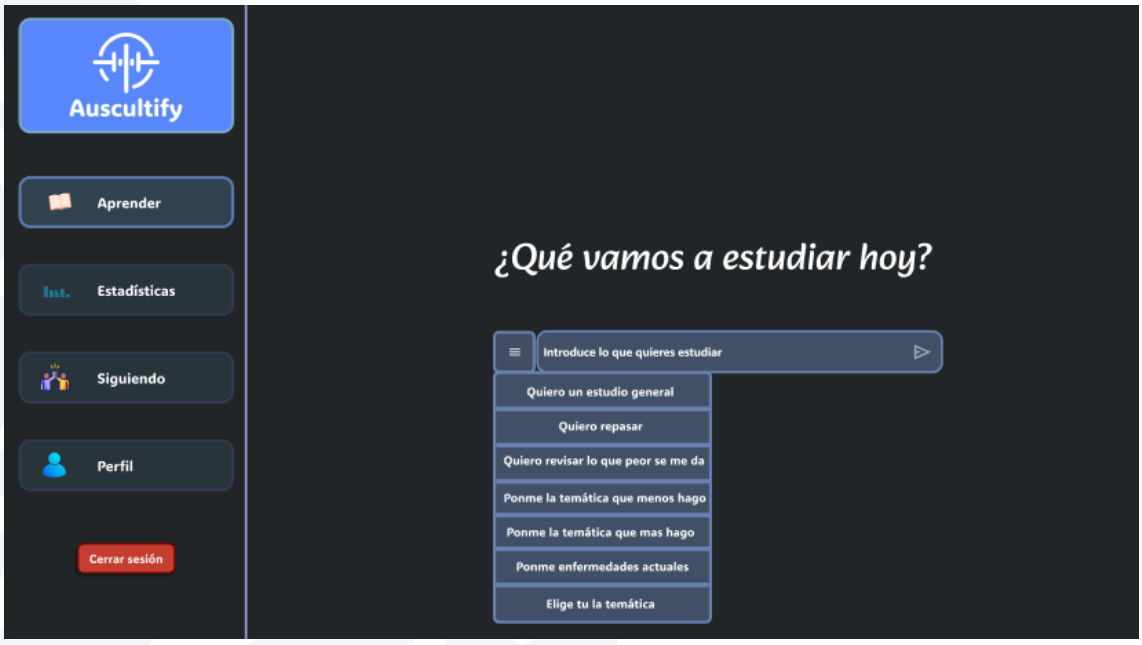


Figura 4.6: Prototipo visual de la pestaña de inicio con desplegable.

Cuando se ha elegido el criterio de selección de preguntas se muestra la figura 4.7, aquí siempre se muestran diez preguntas de las cuales tiene que escucharse un audio y clasificar

qué se cree que es, en caso de fallar lo redirige a la pestaña 4.8, en caso de acertar lo redirige a la figura 4.9.

En la parte superior se ve el progreso de cuántas preguntas se llevan contestadas y cuántas faltan por contestar, se ha decidido que siempre se muestren 10 preguntas. Para escuchar el audio se reproducirá automáticamente y, si necesita volverlo a escuchar, se podrá pulsar el icono de altavoz para que se reproduzca de nuevo. En caso de que no se sepa la pregunta, se podrá saltar y se mostrará como una pregunta fallada, como se ve en la figura 4.8.

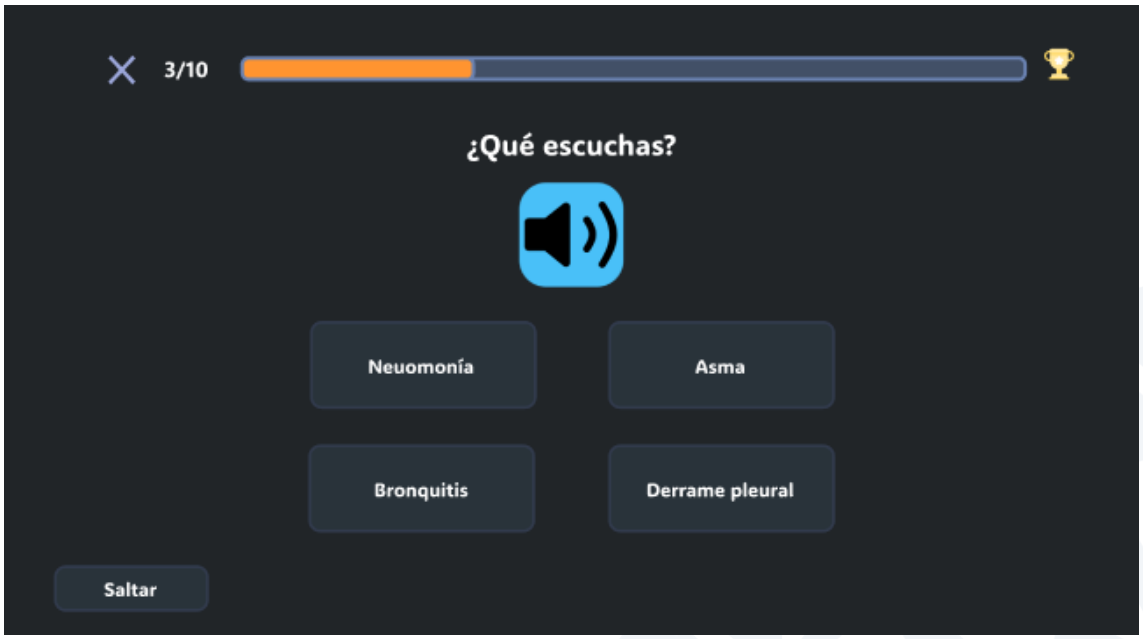


Figura 4.7: Prototipo visual de la pestaña de preguntas.

En la siguiente figura 4.8 se puede ver en caso de elegir una respuesta incorrecta, se muestra la opción que elegiste en rojo, mostrando el error, y muestra cuál debería haber sido la respuesta correcta en la parte inferior, si se pulsa el botón avanzaría a la siguiente pregunta, como se puede ver en la figura 4.7. En caso de haber completado todas las preguntas, se redirigiría a la figura 4.10.



Figura 4.8: Prototipo visual de la pestaña de preguntas cuando te equivocas.

En la siguiente figura 4.9 se puede ver en caso de elegir una respuesta correcta, se muestra la opción que elegiste en verde, mostrando en verde la respuesta correcta, y muestra una felicitación en verde en la parte inferior, si se pulsa el botón avanzaría a la siguiente pregunta, como se puede ver en la figura 4.7. En caso de haber completado todas las preguntas, se redirigiría a la figura 4.10.

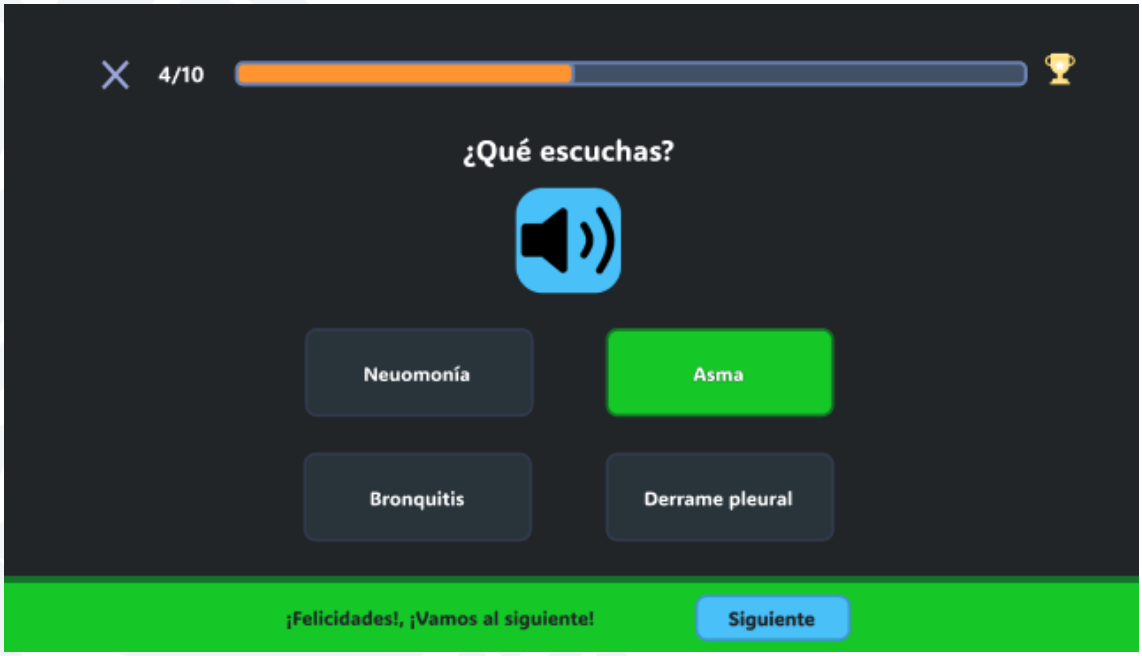


Figura 4.9: Prototipo visual de la pestaña de preguntas cuando aciertas.

En la siguiente figura 4.10 se puede ver lo que se mostraría al acabar la serie de preguntas que te muestra la aplicación, se muestra un recuento de las preguntas acertadas y te permite volver, que te llevaría a la figura 4.5.



Figura 4.10: Prototipo visual de la pestaña de preguntas completadas.

En la siguiente figura 4.11 se pueden ver las estadísticas que tiene un usuario en la aplicación, con distinta información de especial interés, en la aplicación final se ven otros datos y presentado de otra forma.



Figura 4.11: Prototipo visual de la pestaña de estadísticas.

En la siguiente figura 4.12 se puede ver la pestaña siguiendo en el supuesto de que no se siga a nadie en la aplicación. En esta pestaña puedes buscar a otros usuarios que estén usando la aplicación a partir de su correo, como se puede ver en la figura 4.13. Esta pestaña se planteó para darle un enfoque más social a la aplicación para facilitar el aprendizaje y añadir una parte de competición sana entre los usuarios.



Figura 4.12: Prototipo visual de la pestaña de siguiendo sin seguidores.

En la siguiente figura 4.13 se puede ver cómo se vería el buscador en caso de que se quiera añadir una persona para seguir, se mostrará una lista de usuarios que están en la aplicación y que no haya puesto su perfil en privado. Cuando se selecciona algún usuario, se añade a la lista de usuarios a los que sigues como se puede ver en la figura 4.14.



Figura 4.13: Prototipo visual de la pestaña de siguiendo al buscar un usuario.

En la siguiente figura 4.14, se puede ver que se ha añadido un usuario a la lista, si se selecciona, te redirige a la pestaña 4.15, al pulsar la cruz al lado del nombre elimina a ese usuario de la lista y lo redirige de vuelta a 4.12.

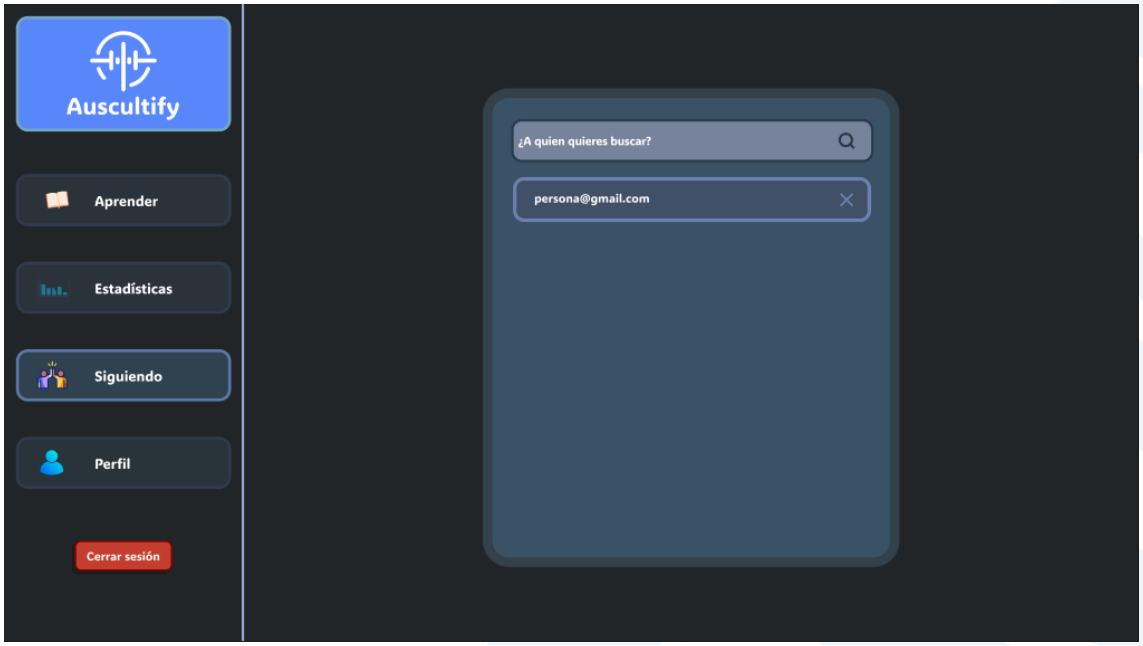


Figura 4.14: Prototipo visual de la pestaña de siguiendo al seguir al menos un usuario.

En la siguiente figura 4.15, se muestra una comparativa entre tú y un usuario al que hayas decidido seguir, es muy parecida a la 4.11, pero esta está pensada con un enfoque más social para que un usuario pueda ver cómo va con la gente que conoce.

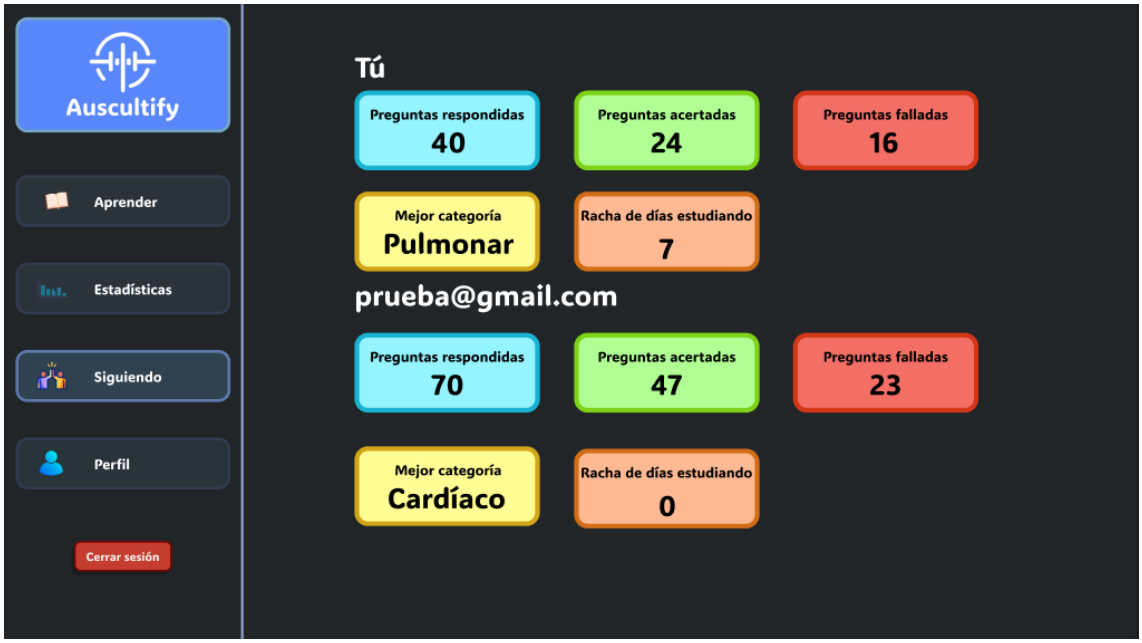


Figura 4.15: Prototipo visual de la pestaña de estadísticas entre ambos usuarios.

Como última sección de la aplicación está el perfil, como se puede ver en la figura 4.16, se puede modificar muchas cosas como el correo electrónico utilizado en la aplicación (siempre y cuando sea un correo válido y no esté siendo usado por otro usuario), la contraseña, ocultar el perfil para que no sea público o eliminar la cuenta. Al pulsar en el lápiz de algunas de estas opciones muestra una serie de cuadros de texto que estaban ocultos, como se puede ver en las figuras 4.17, 4.18 y en el caso de pulsar eliminar cuenta se mostrará un desplegable como se ve en la figura 4.19.



Figura 4.16: Prototipo visual de la pestaña perfil.

A continuación, en la siguiente figura 4.17 se puede ver cómo se vería si se trata de modificar el correo.



Figura 4.17: Prototipo visual de la pestaña perfil al modificar el correo.

A continuación, en la siguiente figura 4.18 se puede ver cómo se vería si se trata de modificar la contraseña.



Figura 4.18: Prototipo visual de la pestaña perfil al modificar la contraseña.

A continuación, en la siguiente figura 4.19 se puede ver cómo se vería si se pulsa eliminar la cuenta, si se pulsa el botón de cancelar se anula el proceso y, en caso de que se vuelva a pulsar eliminar cuenta, se eliminará de la base de datos junto a todo el progreso y datos que tenga un usuario y lo redirigirá a la figura 4.3.



Figura 4.19: Prototipo visual de la pestaña perfil al eliminar una cuenta.

4.4 JUSTIFICACIÓN DE LAS TECNOLOGÍAS UTILIZADAS.

Esta sección sirve para justificar el por qué se han elegido algunas tecnologías frente a otras.

Para el desarrollo de la gran mayoría de la aplicación se ha decidido utilizar TypeScript. Es un lenguaje de programación bastante útil para la elaboración de páginas web. En el caso de por qué se ha priorizado TypeScript frente a JavaScript es debido a que ofrece un tipado estático, es decir, muestra si se tiene un error antes de compilar, agilizando el proceso de desarrollo de la aplicación, mientras que JavaScript muestra el error en tiempo de ejecución.

Hay un ligero inconveniente de utilizar TypeScript frente a JavaScript y es debido a que, en tiempo de compilación, todo código hecho en TypeScript se transforma en JavaScript, haciendo que haya un ligero coste extra a la hora de compilar el programa.

Para el desarrollo backend hay muchas opciones disponibles, y fue en su momento una decisión bastante compleja sobre qué lenguaje utilizar para programar la mayoría de micro-servicios, debido a que se podía haber hecho en Java, Python, C#, etc. Se terminó optando por utilizar Node.js para el desarrollo de los microservicios.

La razón principal se debe a cómo está diseñado en sí Node.js. Este entorno de ejecución es excepcionalmente eficiente para manejar la concurrencia gracias a su modelo asíncrono y de E/S no bloqueante, basado en un bucle de eventos (Event Loop). Este diseño le permite

gestionar miles de conexiones simultáneas sin necesidad de crear múltiples hilos para cada petición, lo que resulta en un mejor rendimiento y mayor escalabilidad para los microservicios. También aporta cohesión general a la aplicación, de esta forma tanto el frontend como el backend estaban programados en frameworks de JavaScript y, por último, también es de los lenguajes más cotizados para el desarrollo web backend, mostrando especial interés en el aprendizaje del mismo.

Como resultado de utilizar Node.js, llevó a buscar una forma de generar los endpoints de los distintos microservicios, para que así el frontend pudiera utilizarlos. Para ello, se utilizó el framework de Express.js, especialmente reconocido para este uso y uno de los más utilizados debido al minimalismo y velocidad que ofrece.

También, aprovechando la estructura de microservicios, se ha optado utilizar otro lenguaje para el backend en el desarrollo. Se ha decidido utilizar Python con el framework de Flask para el microservicio de Gestor Algoritmos. En el presente TFG no se hablará en detalle de por qué se ha utilizado, dicha información se recopila en el CTFG.

Para el desarrollo frontend también hubo muchas opciones disponibles sobre qué lenguaje usar, de la misma manera que para el backend hay decenas de opciones, para el frontend también hay un mundo donde elegir. En el caso del desarrollo de la aplicación se optó por utilizar React.js.

La razón principal de utilizar React.js frente a otras tecnologías fue debido a que ya se partía con conocimientos de desarrollo en React.js. También es una de las tecnologías más prominentes del mercado, siendo de las más utilizadas. Este framework ofrece también beneficios, como poder tener una interfaz declarativa, es decir, en React cuando declaras una interfaz, si esta recibe algún cambio de estado, se actualiza de forma automática, simplificando el desarrollo. Además, un beneficio de utilizar React.js es poder tener acceso a un amplio abanico de documentación gracias a la popularidad del mismo.

Para la creación de la aplicación se ha decidido utilizar el entorno de Visual Studio Code, gracias a lo versátil que es. También, gracias a la cantidad de plugins y extensiones, ofrece una versatilidad que pocos entornos de programación ofrecen. Además, al tener un terminal integrado en el propio Visual Studio Code, permite ejecutar fácilmente la aplicación.

Para la base de datos se ha decidido utilizar MySQL debido a que ya se partía de conocimiento previo de cómo utilizarlo.

4.5 Implementación de las herramientas

En este apartado se detalla cómo se ha implementado y configurado cada parte del proyecto para el desarrollo del mismo.

4.5.1 Preparación del Docker-compose

Se han creado diversos Dockerfiles para toda la aplicación. Se ha creado uno en el frontend con el siguiente contenido:

```
1 FROM node:18-alpine
2
3 WORKDIR /app
```

```
4
5  COPY package*.json ./
6
7  RUN npm ci
8
9  COPY . .
10
11 EXPOSE 5173
12
13 CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Listado 4.1: Dockerfile de frontend

A continuación se muestran los distintos Dockerfiles para el backend, cada uno para cada microservicio utilizado en la aplicación. Solo se pondrá como ejemplo el Dockerfile de iniciar sesión debido a que el resto de Dockerfiles de Node.js siguen una estructura idéntica, cambiando únicamente el puerto en el que se exponen; en este caso el puerto es 3001 y se ha ido incrementando en uno para cada microservicio.

```
1  FROM node:18
2
3  WORKDIR /app
4
5  COPY package*.json ./
6
7  RUN npm install
8
9  COPY . .
10
11 RUN npm run build
12
13 EXPOSE 3001
14
15 CMD ["npm", "start"]
```

Listado 4.2: Dockerfile de microservicio

Para el microservicio de GestorAlgoritmos, como está programado en Python, se ha hecho un Dockerfile diferente. No sigue el mismo patrón que los otros microservicios; a continuación se puede ver cómo es.

También se ha creado el siguiente Docker-compose para ejecutar los distintos Dockerfiles y hacer que se ejecuten los distintos componentes y servicios de la aplicación.

```
1  FROM python:3.11-alpine
2
3  WORKDIR /app
4
5  COPY requirements.txt .
6
7  RUN pip install --no-cache-dir -r requirements.txt
8
9  COPY . .
10
11 EXPOSE 3014
12
13 CMD ["python", "app.py"]
```

Listado 4.3: Dockerfile de GestorAlgoritmos

4.5.2 Creación del proyecto Frontend

Se ha optado por utilizar React para la configuración inicial del proyecto debido a la versatilidad de dicho framework. Para configurar el proyecto frontend de manera inicial hay que poner en el terminal el siguiente comando.

```
npm create vite@latest Auscultify -- --template react-ts
```

Al ejecutarlo te deja elegir diversas opciones, se ha elegido el framework React con la variante TypeScript. Lo bueno de utilizar TypeScript frente a JavaScript es que te muestra los errores antes que JavaScript, debido a que JavaScript solo te los detectaría en tiempo de ejecución y también, debido a que en TypeScript te obliga a poner de forma explícita los tipos de las variables, se reducen las probabilidades de errores frente a JavaScript.

Posteriormente, para instalar las dependencias necesarias, hace falta poner el siguiente comando en el terminal.

```
npm install
```

Por último, si se desea ejecutar solo la parte frontend del proyecto, lo cual es especialmente útil durante la fase de maquetado sin lógica, hay que poner el siguiente comando, que por defecto despliega la aplicación en el puerto 5173.

```
npm run dev
```

También se han descargado las dependencias necesarias para que el proyecto funcione, se ha puesto el siguiente comando en el terminal para ello.

```
npm install react-router-dom
```

4.5.3 Creación del proyecto Backend

Para cada microservicio del backend se ha creado un directorio para cada uno; cada directorio está compuesto por un Dockerfile que posteriormente se integra con el resto de servicios con el Docker-compose. Por lo general, se ha usado en la implementación Node.js debido a la versatilidad que ofrece dicho lenguaje y a la especialización del autor del presente documento, aunque hay microservicios programados en Python.

Para los microservicios programados en Node.js se han utilizado los siguientes comandos:

```
npm init -y
npm install express bcryptjs cors mysql2
npm install --save-dev typescript @types/express @types/node
@types/jsonwebtoken @types/bcryptjs @types/cors ts-node nodemon
npm install
```

Aparte de los anteriores comandos, se ha creado un archivo llamado tsconfig.json para cada microservicio que contiene el código a continuación y sirve para compilar los archivos TypeScript; todos tienen la misma estructura.

```
1 {
2   "compilerOptions": {
```

```
    "target": "es2019",
    "module": "commonjs",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true
  },
  "include": ["*.ts"]
}
```

Listado 4.4: Codigo de tsconfig.json

También, además del archivo anterior, se ha creado otro archivo llamado index.ts; este archivo contiene toda la funcionalidad del microservicio en sí y se detallará cuando se explique cada microservicio.

5 Implementación

5.1 Frontend

Este apartado se centra en detallar toda la implementación de la aplicación desde el punto de vista Frontend; en cada uno de los apartados se mostrará una imagen visual de la aplicación junto al HTML que se ha creado para obtener ese resultado y su CSS correspondiente. También se mostrarán algunas funciones de lógica en función de si dicha vista tiene o no métodos incorporados.

5.1.1 Inicio de sesión

Este componente se utiliza para toda la visualización de inicio de sesión y que así el usuario pueda entrar en la aplicación. A continuación en la siguiente figura 5.1 se muestra cómo se ve visualmente la aplicación en el estado actual. En la aplicación actual se muestran diversos mensajes de error, entre los cuales se encuentra un mensaje de error por si te has equivocado a la hora de poner el correo o la contraseña.

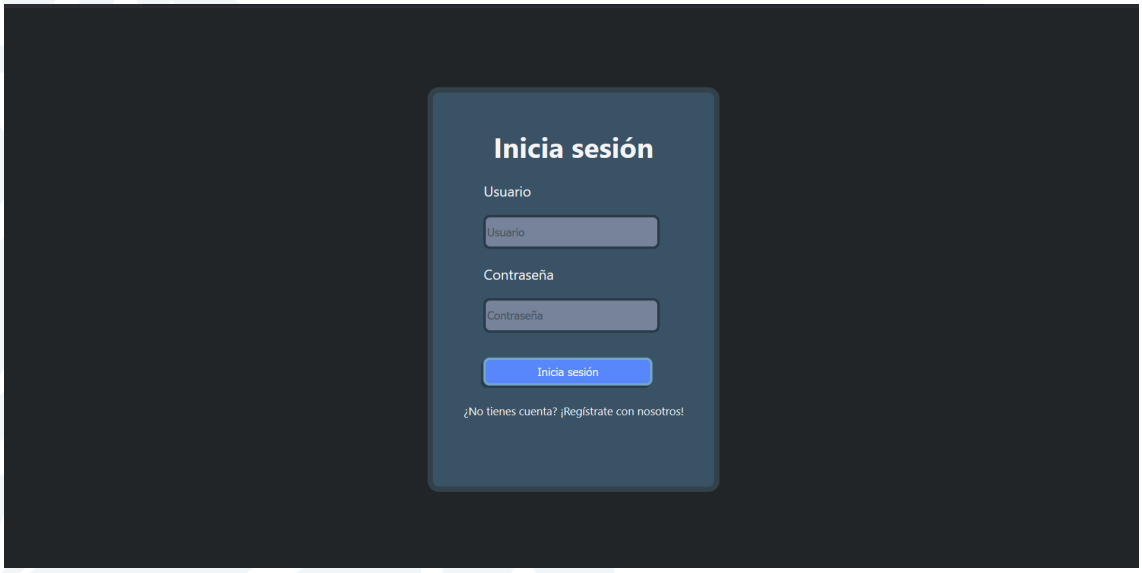


Figura 5.1: Iniciar sesión final.

Cuando se accede a esta pestaña, se llama automáticamente a un endpoint del backend que se llama SincronizarDatos. Este microservicio se encarga de crear automáticamente un

admin en la aplicación si no lo hubiera, y se encarga de crear las categorías de las preguntas en función de lo subido en el repositorio.

Si se pulsa el "¿No tienes cuenta? ¡Regístrate con nosotros!", redirige al usuario a la figura 5.2. Si se introducen los datos del usuario, si es admin lo redirige a la figura **??**, si no, redirige al usuario a la figura 5.3.

5.1.2 Registrarse

Esta pestaña se utiliza para toda la visualización de registrarse y que así el usuario pueda crearse una cuenta en la aplicación. A continuación, en la siguiente figura 5.2. Actualmente se muestran diversos mensajes de error si introduces un correo que no es válido, si no coinciden las contraseñas, si existe un usuario con el mismo correo, o si falta por introducir algún campo.

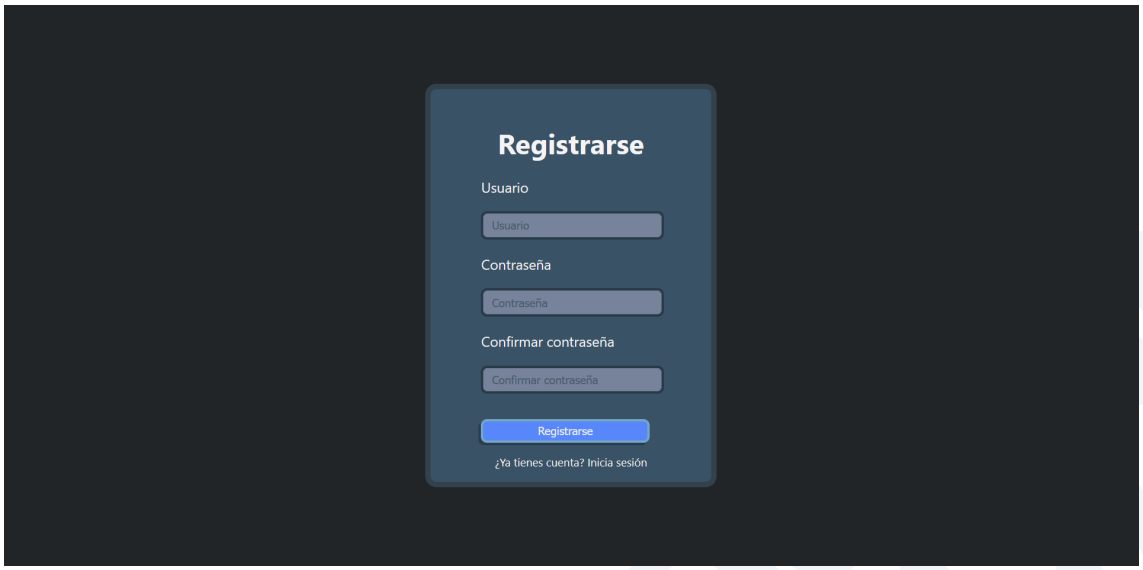


Figura 5.2: Registrarse final.

Si se pulsa "¿Ya tienes cuenta? Inicia sesión", redirige al usuario a la figura 5.1. Si se introducen datos válidos de usuario, redirige al usuario a la figura 5.3.

5.1.3 Pestaña de inicio usuario normal

Esta pestaña se utiliza como núcleo central de la aplicación, como se puede ver en la figura 5.3, hay un menú a la izquierda con distintas opciones, y en la parte de la derecha un recuadro donde se puede elegir qué algoritmos quieres estudiar.



Figura 5.3: Inicio final.

El menú de la izquierda es un componente que se reutiliza en toda la aplicación y lo que va variando es el contenido de la derecha en función de la navegabilidad del usuario.

Si se hace clic en el botón Aprender a la izquierda, redirige al usuario a esta misma figura 5.3, si se hace clic en estadísticas, redirige al usuario a la figura 5.7, si se hace clic en siguiendo, redirige al usuario a la figura 5.9, si se hace clic en perfil, redirige al usuario a la figura 5.14.

Cuando se hace clic en el desplegable central, se termina abriendo el menú de los algoritmos, como se puede ver en la figura 5.4.

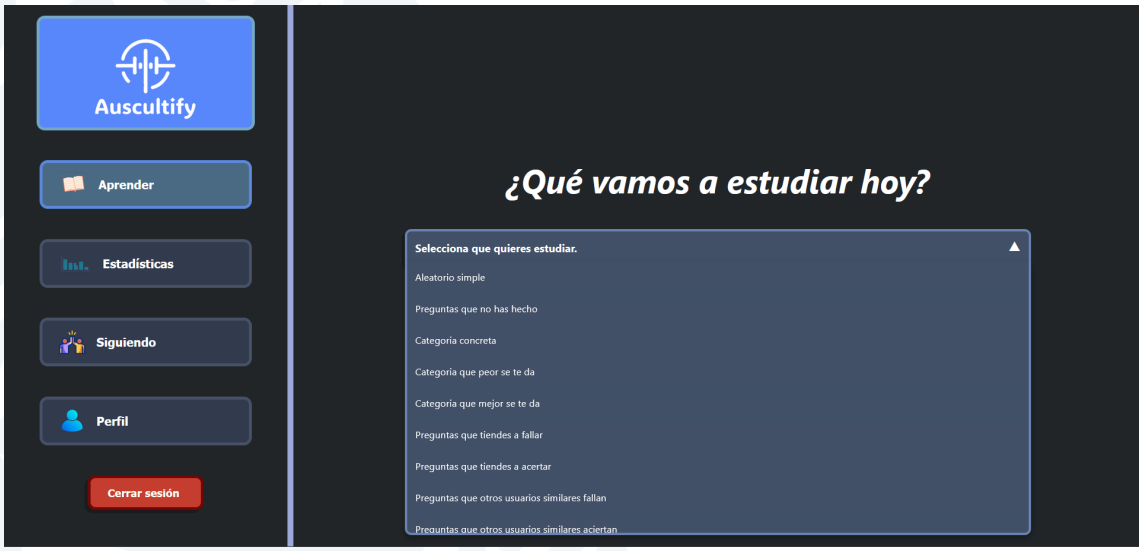


Figura 5.4: Inicio con el desplegable abierto final.

También, si se pasa el cursor por algún algoritmo en concreto, muestra la descripción de dicho algoritmo para que de esta forma el usuario sepa qué opción está eligiendo, como se puede ver en la figura 5.5.



Figura 5.5: Inicio con el desplegable abierto y con una descripción final.

Por último, si se hace clic en el algoritmo Categoría concreta, se añade otro desplegable para elegir qué categoría quieres estudiar, como se puede ver en la figura 5.6.



Figura 5.6: Inicio con el desplegable de las categorías abierto.

Si se hace clic en cualquiera de los algoritmos o en alguna categoría en concreto, redirige al usuario a la figura 5.19.

5.1.4 Estadísticas individuales

Esta pestaña se utiliza para ver tu progreso en la aplicación, se puede observar en la siguiente figura 5.7, donde se puede ver las preguntas que has respondido en total, cuántas has acertado, cuántas has fallado, el número de días seguidos que llevas estudiando, tu promedio general a la hora de acertar o fallar preguntas. Cuál es la categoría que mejor se te da estudiar, el promedio de aciertos de dicha categoría y el promedio de aciertos de la peor categoría.

También hay un gráfico semanal con tu progreso a diario, cuántas preguntas has respondido ese día, cuántas has acertado y cuántas has fallado.

Esta pestaña es especialmente útil para que el usuario vea una sensación de progreso y se motive en querer seguir aprendiendo, en mejorarse a sí mismo cada día.



Figura 5.7: Estadísticas de un usuario.

Si se pone el cursor encima de alguna de las líneas de la gráfica, como se puede ver en la figura 5.8, muestra el número exacto en detalle respecto a cuánto has hecho ese día.



Figura 5.8: Estadísticas de un usuario donde se ve la información exacta.

5.1.5 Siguiendo

Esta pestaña se utiliza para seguir a otros usuarios que utilicen la aplicación. Solo aparecen los usuarios que tienen puesto su perfil público. Como se puede ver en la figura 5.9. No se muestra ningún usuario si no lo sigues, solo se muestra un texto instándote a que sigas a otro usuario.

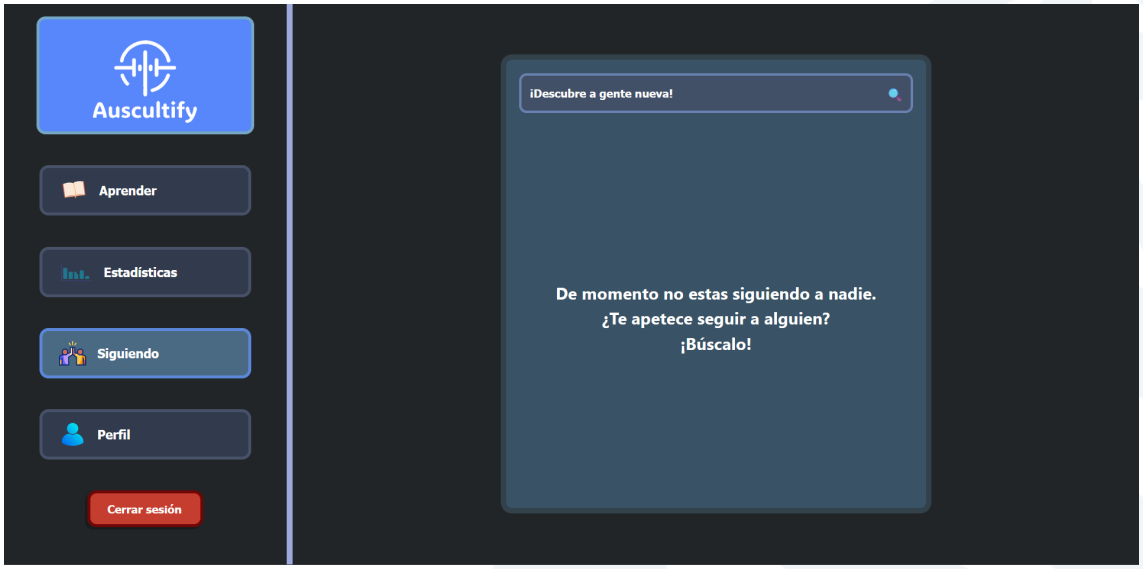


Figura 5.9: Pestaña de siguiendo sin seguir a otro usuario.

Para buscar algún usuario hace falta hacer clic en el buscador en la parte superior, donde al introducir el inicio de un correo, se muestra una lista de usuarios coincidentes, como se puede ver en la figura 5.10.

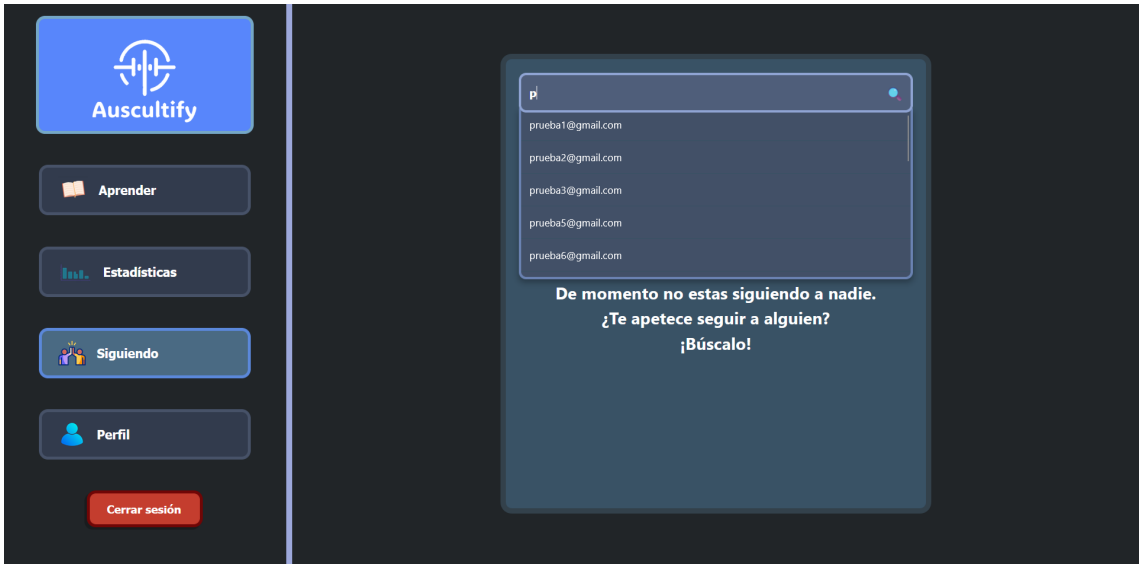


Figura 5.10: Pestaña de siguiendo al buscar un usuario.

Si se busca algún usuario que no está en la aplicación, se muestra como resultado el siguiente, un mensaje de error diciendo que no hay ningún usuario coincidente, como se puede ver en la figura 5.11

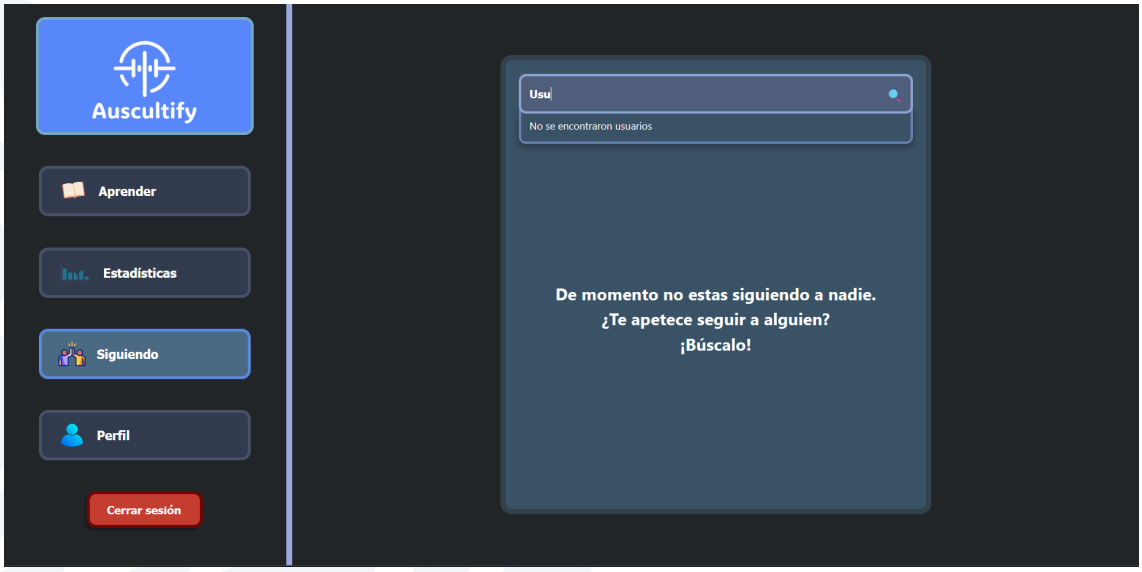


Figura 5.11: Pestaña de siguiendo al buscar un usuario que no existe.

Si seguimos a algún usuario, este desaparece de la lista del buscador y se modifica el recuadro para mostrar a los usuarios que sigues, como se puede ver en la figura 5.12. En esta lista se pueden hacer varias cosas, si se hace clic en alguno de estos usuarios, redirige al estudiante a la figura 5.27.

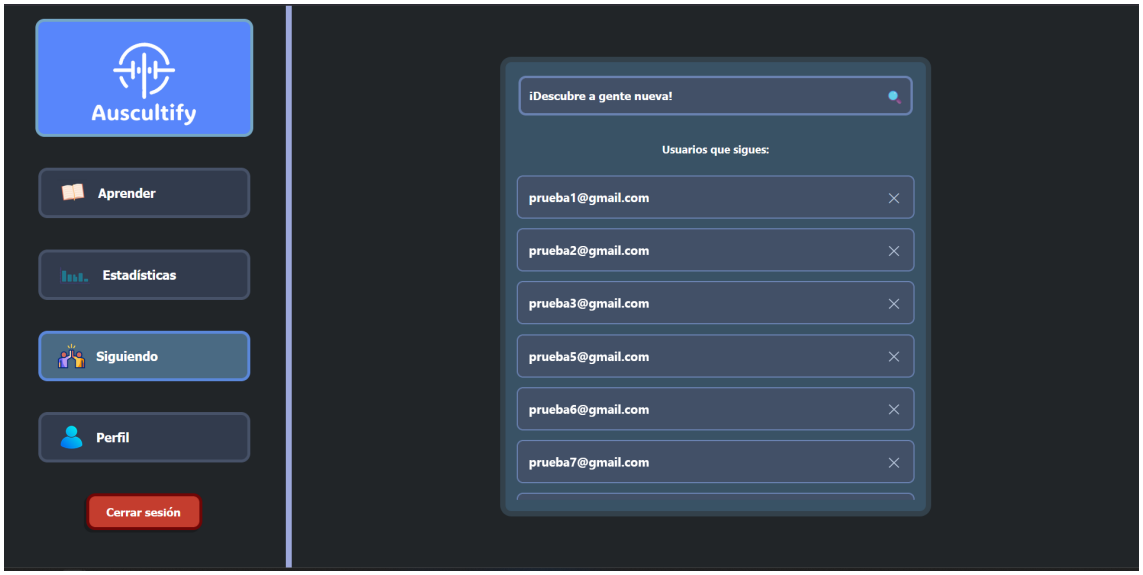


Figura 5.12: Pestaña de siguiendo si se siguen usuarios.

También se puede eliminar algún usuario en concreto pulsando la x a la derecha, como se puede ver en la figura 5.13.

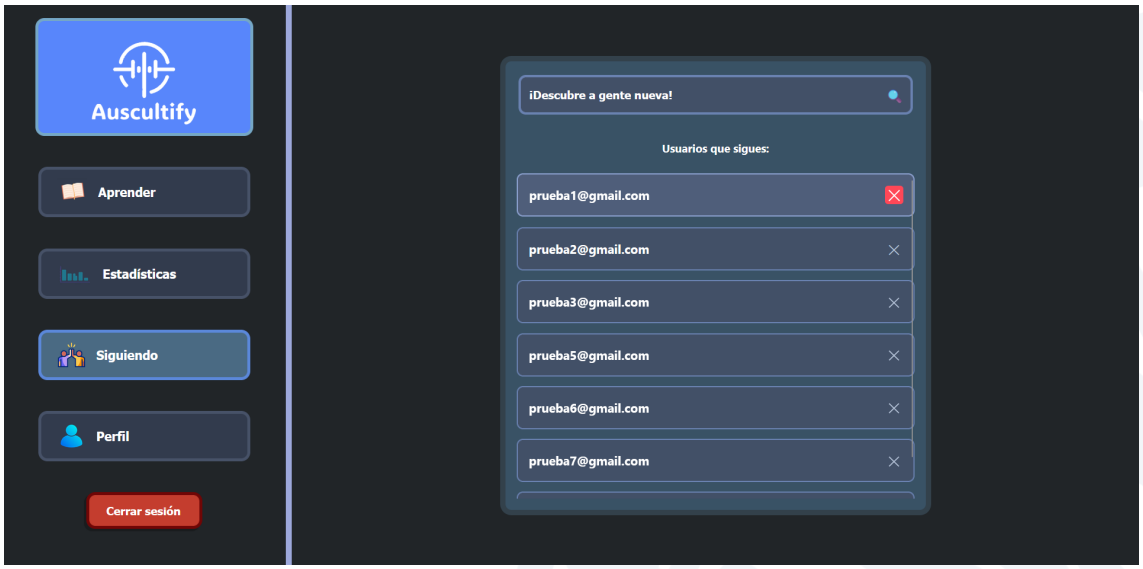


Figura 5.13: Pestaña de siguiendo si se desea eliminar algún usuario.

5.1.6 Perfil

Esta pestaña sirve para modificar los datos del usuario, entre ellos el correo que se ha usado, la contraseña, si quiere ocultar el perfil o si quiere eliminar la cuenta, como se puede ver en la figura 5.14.



Figura 5.14: Perfil final.

Si se hace clic en el lápiz del correo electrónico se muestra un cuadro de texto para modificar el correo, como se puede ver en la figura 5.15. Saltan mensajes de error en el caso de que se introduzca un correo electrónico inválido, o se intente usar un correo que otro usuario en la aplicación esté usando. Si estuviera abierto el cuadro de texto de la contraseña, esta se cierra y muestra el cuadro de texto del correo.

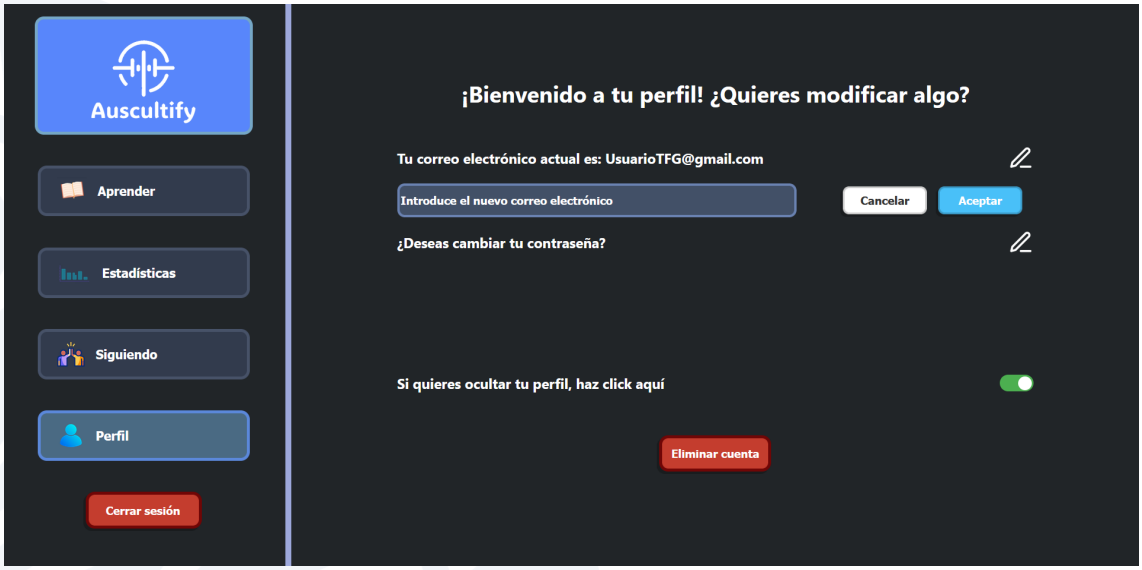


Figura 5.15: Perfil si se desea modificar el correo.

Si se hace clic en el lápiz de la contraseña se muestran dos cuadros de texto para modificar la contraseña, como se puede ver en la figura 5.16. Saltan mensajes de error en el caso de que se introduzcan dos contraseñas que no coinciden. Si estuviera abierto el cuadro de texto del correo, esta se cierra y muestra el cuadro de texto de la contraseña.



Figura 5.16: Perfil si se desea modificar la contraseña.

Si se decide hacer el perfil privado, se ve de la siguiente forma en la figura 5.17



Figura 5.17: Perfil privado.

Si se desea eliminar la cuenta, al pulsar el botón eliminar cuenta se muestra un desplegable preguntando si se desea de verdad eliminar la cuenta o cancelar dicha acción, como se puede ver en la figura 5.18. Si se elimina la cuenta, se pierden todas las estadísticas asociadas, y se elimina de la lista de seguidores si algún usuario lo seguía.

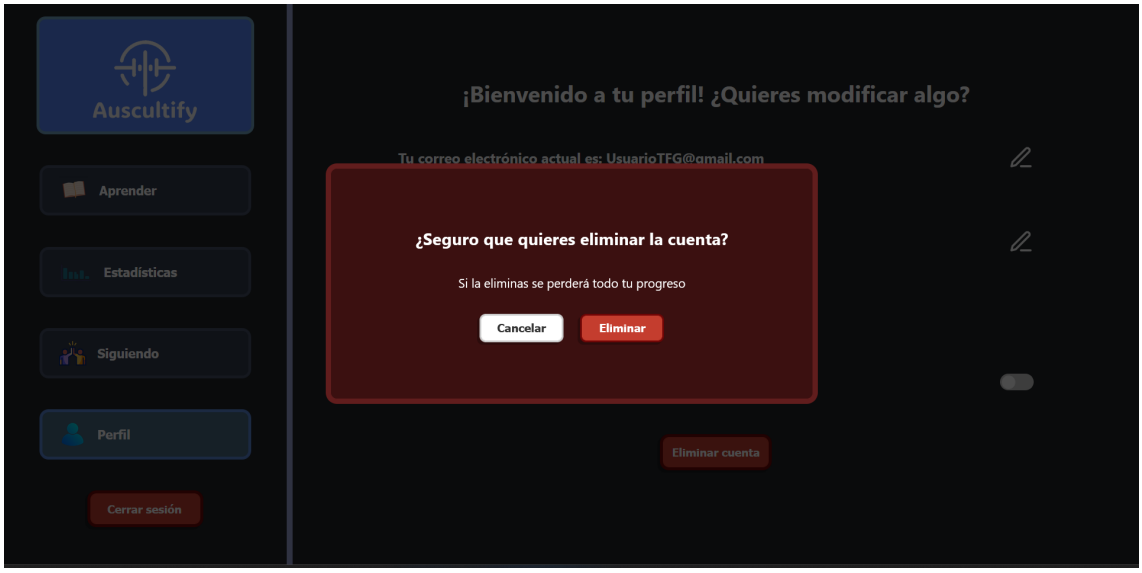


Figura 5.18: Mensaje de confirmación de eliminación de cuenta del perfil.

5.1.7 Responder preguntas

A esta pestaña se accede si has elegido algún algoritmo de la pestaña de inicio, como se puede ver en la figura 5.19. Cuando se accede, se cargan 10 preguntas en base a los criterios de los distintos algoritmos utilizados, en la parte superior se puede ver el progreso a la hora de responder las preguntas mediante una barra. También si se pulsa el altavoz, reproduce el audio a la pregunta asociada. Hay 4 opciones para elegir sobre qué puede ser ese audio, y si no se supiera cuál es, se puede saltar dicha pregunta y pasar a la siguiente.



Figura 5.19: Pestaña para responder una pregunta.

Si se falla una pregunta, se muestra que el usuario se ha equivocado poniendo dicha opción en rojo y mostrando cuál hubiera sido la respuesta correcta, como se puede ver en la figura 5.20. Si se pulsa el botón de siguiente, redirige al usuario a la figura 5.19.



Figura 5.20: Pestaña para responder una pregunta fallida.

Si se acierta una pregunta, se muestra que el usuario ha acertado y se pone dicha opción en verde, también felicita al usuario por haber acertado, como se puede ver en la figura 5.21. Si se pulsa el botón de siguiente, redirige al usuario a la figura 5.19.

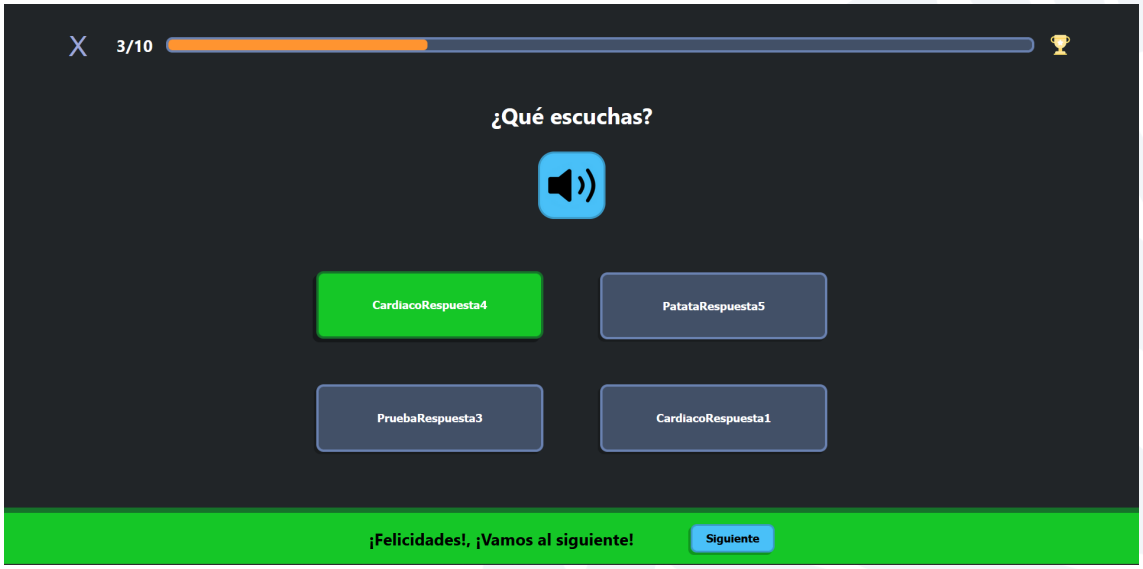


Figura 5.21: Pestaña para responder una pregunta acertada.

Cuando se contestan las 10 preguntas, hay 3 variantes al final de todo. Estas variantes son la pestaña de resultados, si aciertas menos de 5, se muestra la figura 5.22. Si se acierta

más de 5 hasta 9 se muestra la figura 5.23. Y por último si se aciertan todas se muestra la siguiente figura 5.24.

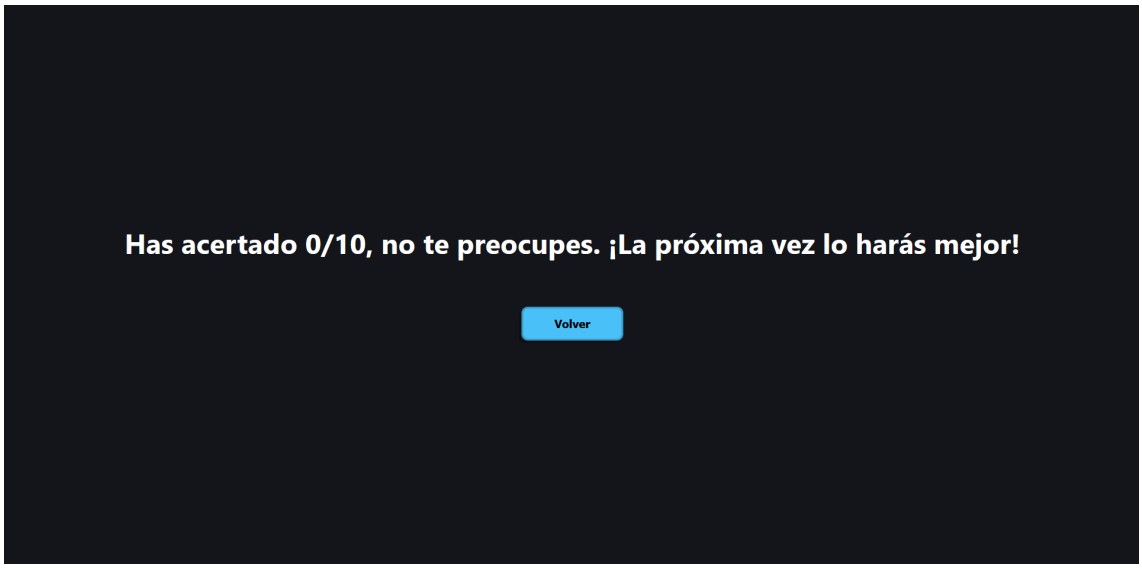


Figura 5.22: Pestaña de resultados si has acertado menos de 5.

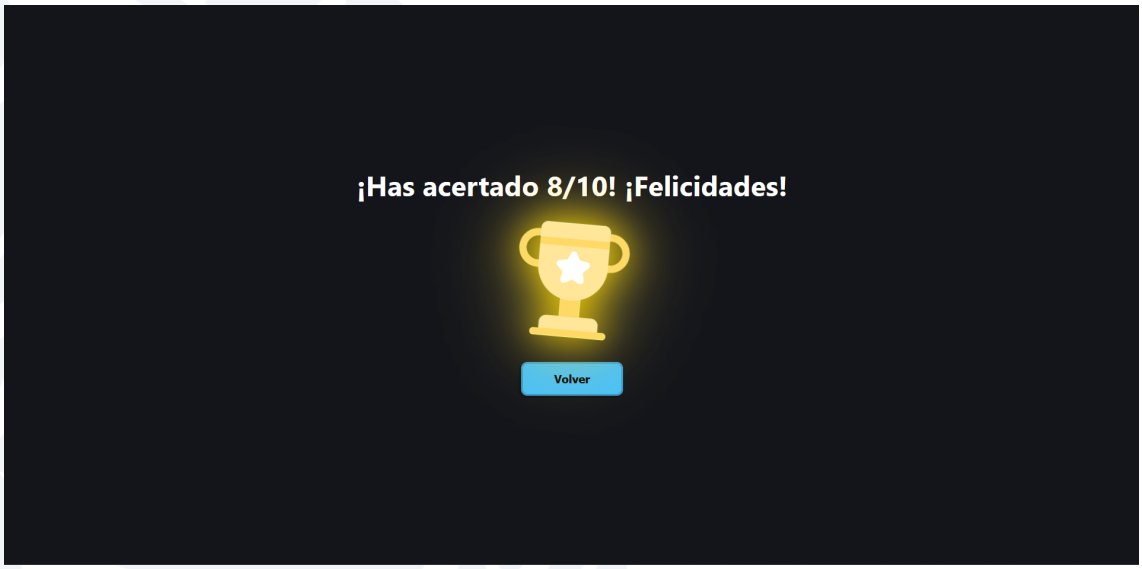


Figura 5.23: Pestaña de resultados si has acertado más de 5 y menos de 9.



Figura 5.24: Pestaña de resultados si has acertado todas las preguntas.

5.1.8 Perfil admin

Esta pestaña es diferente respecto al perfil normal, como se puede ver en la figura 5.25. La opción de modificar el correo está deshabilitada, no se puede modificar el correo electrónico. Tampoco se puede eliminar la cuenta del administrador.

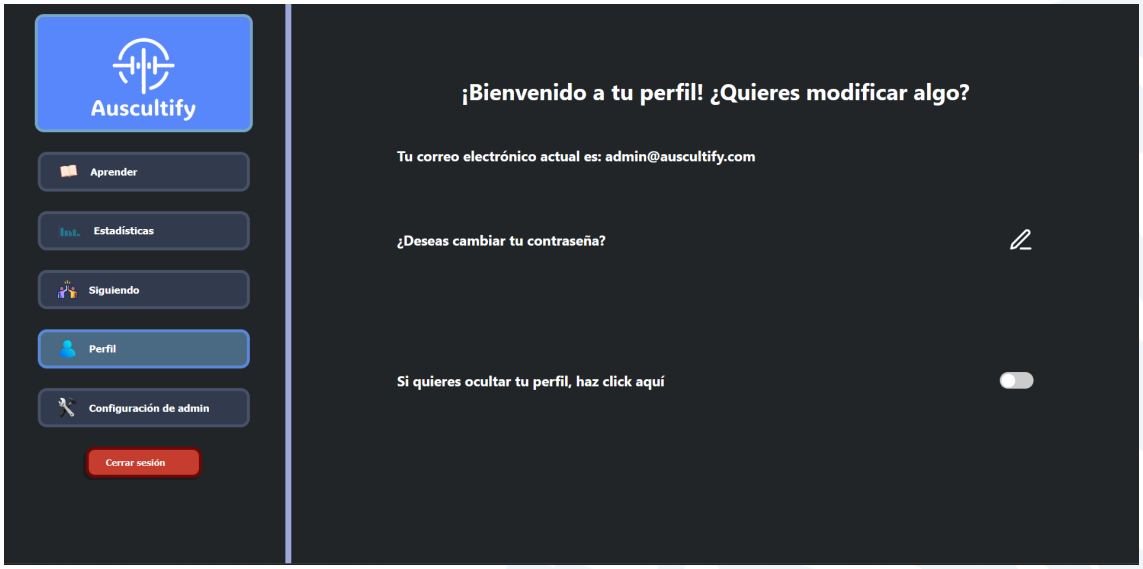


Figura 5.25: Perfil del admin.

5.1.9 Panel admin

Esta pestaña solo se puede acceder si eres el administrador de la aplicación, como se puede ver en la figura 5.26. Hay diversas funciones imprescindibles para la aplicación como poder crear una pregunta en la parte superior, en la parte central crear una nueva categoría

o en la parte inferior eliminar una categoría o pregunta. También, si nos fijamos en la parte de la izquierda, en el menú se ha añadido un botón que es la configuración del admin, este botón está oculto para los usuarios que no son admin y sirve para acceder a esta pestaña.



Figura 5.26: Panel del admin.

5.1.10 Comparativa usuarios

A esta pestaña se accede a partir de la lista de siguiendo si se hace clic en algún usuario al que sigas, como se puede ver en la figura 5.27. Sirve principalmente para ver tu progreso en función de los datos de tu amigo. Esta pestaña es especialmente importante porque refuerza la idea de competitividad y mejoría entre usuarios.

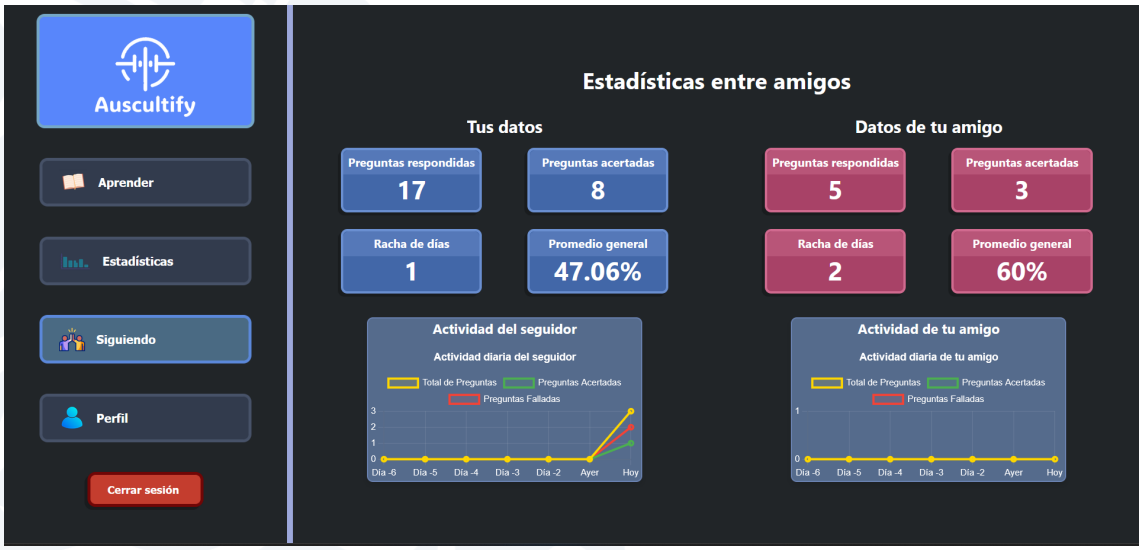


Figura 5.27: Panel del admin.

5.2 BACKEND

Este apartado se centra en detallar toda la implementación de la aplicación desde el punto de vista Backend, en cada uno de los apartados se mostrarán los directorios diferentes para cada microservicio programado.

Se hará caso omiso al microservicio relacionado con la gestión de los algoritmos, debido a que se explica en detalle en el CTFG.

5.2.1 Sincronizar datos.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3001, se puede acceder a él mediante el siguiente endpoint:

- /**sincronizar-datos**: Endpoint que se encarga de crear un admin si no lo hay y crear los directorios de las categorías según un .txt interno y añadir en la base de datos si no existiera esa categoría.

Este microservicio se creó para el arranque inicial de la aplicación, se asegura de que existe un admin y si no lo hay lo crea. Dicho microservicio se ejecuta al acceder a la pestaña de iniciar sesión, que es la primera pestaña que se ejecuta de la app.

Además de crear el admin, se asegura de que existen los directorios de las categorías y si las categorías están en la base de datos. Esto es especialmente útil para asegurar la consistencia de datos entre dispositivos, es una forma de asegurar la sincronización si se baja el repositorio.

La lógica detrás de la sincronización es revisar un txt donde se almacenan las distintas categorías, y se comprueba si existe un directorio con dicho nombre. También revisa en la base de datos si hay o no una categoría así.

5.2.2 Gestión autentificación.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3002, se puede acceder a él mediante los siguientes endpoints:

- /**registrarse**: Es un endpoint el cual, si se le introduce una dirección de correo y dos contraseñas, termina generando un usuario en el sistema si no existe dicho usuario. Internamente comprueba si el correo es una dirección válida y si las contraseñas coinciden. Si se cumplen esos requisitos se encripta la contraseña y se comprueba si existe el usuario. Si no existe el usuario, se añade a la base de datos con los datos proporcionados. Como resultado, devuelve el usuario que se ha creado.
- /**login**: Es un endpoint el cual, si se le introduce una dirección de correo electrónico y una contraseña, comprueba si hay algún usuario con la contraseña coincidente y el correo coincidente. Si es así, devuelve el usuario correspondiente.

Este microservicio se creó para todas las tareas relacionadas con la autentificación tanto para el login como para registrar un usuario. Se usa en la pestaña correspondiente de

registrarse e iniciar sesión. Este microservicio es vital para el correcto funcionamiento de la aplicación.

5.2.3 Gestión perfil.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3003, se puede acceder a él mediante los siguientes endpoints:

- /**modificar-perfil**: Es un endpoint el cual, si se le introduce un id de usuario y algún dato que se desea modificar, ya sea si se quiere modificar si es público, el correo o la contraseña. No es obligatorio ninguno de los campos salvo el id. Dentro del método se comprueba qué es lo que se quiere modificar en función de si el parámetro es distinto de null. Devuelve el usuario entero con los datos actualizados.
- /**eliminar-cuenta**: Es un endpoint el cual, si se le introduce un correo electrónico y una contraseña, elimina algún usuario de la base de datos. Dentro del método se comprueba si existe un usuario con dichos datos y si existe, se elimina. Sólo devuelve un mensaje de éxito.

Este microservicio se utiliza dentro de la pestaña del perfil. Si bien en el frontend para modificar algo del usuario no puedes hacerlo todo a la vez, es decir, si quieres modificar el correo no puedes estar modificando la contraseña, realmente en el backend sí permitiría varios cambios pero se tomó la decisión de que en el frontend no se pudiera hacer varios cambios simultáneos para asegurar la minimización de posibles errores.

5.2.4 Gestión siguiendo.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3004, se puede acceder a él mediante los siguientes endpoints:

- /**obtener-usuarios-publicos**: Es un endpoint el cual, si se le introduce un parámetro de búsqueda y el correo electrónico del usuario actual, sirve para obtener todos los usuarios con el tag de que es público en la base de datos filtrados. Estos usuarios públicos se les filtra en función a 4 criterios.

  El primero es que dicho usuario sea público.

  El segundo, se excluye el usuario actual de la búsqueda, no sale el usuario que le hayas pasado.

  El tercero es que no salen los usuarios que ya siga el usuario actual.

  El cuarto es que se filtra en función al criterio de búsqueda, solo muestra los usuarios que comiencen por esa cadena de caracteres.

  El resultado de este endpoint es un array de los correos que cumplan estas condiciones.
- /**seguir**: Es un endpoint el cual, si se le introducen dos correos electrónicos, hace que el primer correo siga al segundo correo. Se hacen diversas comprobaciones para evitar que se siga a sí mismo o que se intente seguir a un usuario que ya sigues o si se

intenta seguir un usuario que no existe. Se añade en la base de datos dicha relación y devuelve como resultado un mensaje de éxito.

- /**obtener-siguiendo**: Es un endpoint el cual, si se le introduce un correo electrónico, busca todos los usuarios que sigue ese correo que sean públicos y devuelve un array con los correos que sigue ese usuario.

- /**eliminar-siguiendo**: Es un endpoint el cual, si se le introducen dos correos electrónicos, hace que el primer correo deje de seguir al segundo correo. Se hacen diversas comprobaciones para evitar que se intente dejar de seguir a un usuario que no sigues o si se intenta dejar de seguir un usuario que no existe y devuelve como resultado un mensaje de éxito.

Este microservicio se utiliza dentro de la pestaña de seguir. Este microservicio se encarga de todas las operaciones relacionadas con esa función.

5.2.5 Gestión categoría.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3005, se puede acceder a él mediante los siguientes endpoints:

- /**crear-categoria**: Es un endpoint el cual, si se le introduce el nombre de una categoría, hace que se cree un directorio en el proyecto y se añade en la base de datos. El nombre no debe de coincidir con alguna otra categoría ya existente, tiene que tener al menos 2 caracteres y máximo 50, además de no usar caracteres especiales. Una vez se crea una categoría, se actualiza un txt interno que sirve para asegurar la consistencia entre repositorios.

- /**obtener-categorias**: Es un endpoint el cual, si se le llama, devuelve todas las categorías almacenadas en la base de datos.

- /**eliminar-categoria**: Es un endpoint el cual, si se le introduce el id o el nombre de una categoría, elimina el directorio (aun si tiene preguntas dentro), elimina las preguntas asociadas a dicha categoría si las hubiera de la base de datos, elimina la categoría de la base de datos y elimina dicha categoría del txt. Devuelve un mensaje de éxito y los datos eliminados.

Este microservicio se utiliza para todas las tareas relacionadas con la obtención, creación y eliminación de las categorías que el admin utiliza en su panel. Este microservicio modifica el scaffolding del proyecto de forma directa para poder ir almacenando las preguntas asociadas a cada categoría de una manera organizada.

5.2.6 Gestionar preguntas.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3012, se puede acceder a él mediante los siguientes endpoints:

- /**crear-pregunta**: Es un endpoint el cual, si se le introduce una categoría, una respuesta y un archivo de audio, hace que se almacene ese audio en el directorio asociado a esa categoría. Se comprueba que se han introducido todos los campos, si la categoría existe, si encuentra el directorio y se almacena la pregunta en la base de datos si se cumplen esas condiciones. También, cuando se almacena el audio, se actualiza la ruta del audio en un txt interno. Cuando se almacena la pregunta devuelve todos los datos asociados a dicha pregunta.

- /**obtener-preguntas**: Es un endpoint el cual se le puede introducir o no una categoría y devuelve las preguntas con los datos correspondientes, si no se le pone una categoría concreta devuelve todas las preguntas de la base de datos.

- /**eliminar-pregunta**: Es un endpoint el cual, si se le introduce un id de la pregunta, la elimina. También elimina la pregunta del repositorio y elimina la ruta de la pregunta del txt. Al eliminar la pregunta devuelve el archivo de audio y un mensaje de éxito.

Este microservicio se utiliza para todas las tareas relacionadas con la obtención, creación y eliminación de las preguntas que el admin utiliza en su panel. Este microservicio modifica el scaffolding del proyecto de forma directa para poder ir almacenando las preguntas asociadas a cada categoría de una manera organizada.

5.2.7 Obtener algoritmos.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3013, se puede acceder a él mediante el siguiente endpoint:

- /**obtener-algoritmos**: Es un endpoint el cual devuelve todos los datos asociados de un algoritmo.

Este microservicio se utiliza en la pestaña de inicio para obtener los datos de los distintos algoritmos que se pueden elegir en la aplicación.

5.2.8 Gestor datos usuario preguntas.

Este microservicio está programado en Node.js. Está desplegado en el puerto 3015, se puede acceder a él mediante los siguientes endpoints:

- /**actualizar-datos-usuario**: Es un endpoint que permite actualizar las estadísticas de un usuario en función de las respuestas proporcionadas a las preguntas. Se le debe proporcionar el ID del usuario, un array con los resultados de las preguntas (indicando si fueron acertadas o no) y, opcionalmente, la fecha de contestación. Este endpoint actualiza las estadísticas generales del usuario, como el total de preguntas acertadas, falladas, contestadas, la racha actual y el último día de actividad. Además, registra cada respuesta en la base de datos. Devuelve un mensaje de éxito junto con las estadísticas actualizadas del usuario y las estadísticas de la sesión.

- **/estadisticas-usuario/:usuariold**: Es un endpoint que permite obtener las estadísticas generales y por categoría de un usuario. Se le debe proporcionar el ID del usuario como parámetro. Devuelve datos como el porcentaje general de aciertos, la categoría más usada, el porcentaje de aciertos en la mejor y peor categoría, y las estadísticas de preguntas contestadas, acertadas y falladas en los últimos 7 días. También incluye las estadísticas generales del usuario, como el total de preguntas acertadas, falladas, contestadas y la racha actual.
- /**obtener-usuario-correo**: Es un endpoint que permite obtener información básica de un usuario a partir de su correo electrónico. Se le debe proporcionar el correo como parámetro de consulta. Devuelve datos como el ID del usuario, el total de preguntas contestadas, acertadas, la racha actual y el promedio general de aciertos. Si el correo no está registrado, devuelve un mensaje indicando que el usuario no fue encontrado.

Este microservicio se utiliza para gestionar las estadísticas y datos de los usuarios relacionados con las preguntas contestadas en la aplicación. Es esencial para el seguimiento del rendimiento de los usuarios y para proporcionar estadísticas detalladas en la interfaz de usuario.

6 Conclusión y trabajo futuro.

6.1 Conclusiones

A lo largo de este Trabajo de Fin de Grado se ha diseñado y desarrollado una aplicación web orientada a la enseñanza de la auscultación para estudiantes de medicina. La aplicación permite practicar el reconocimiento de sonidos corporales a partir de audios previamente almacenados y organizados por categorías, presentados en forma de preguntas tipo test. Para hacerlo posible, se ha definido una arquitectura clara que separa la capa de presentación del resto de servicios.

En el plano técnico, se ha implementado un frontend en React que ofrece una interfaz sencilla y agradable, con distintas vistas para el inicio de sesión, el registro, la realización de preguntas, la visualización de estadísticas y la comparación entre usuarios. En paralelo, se ha construido un backend basado en una arquitectura de microservicios, donde cada servicio se encarga de una responsabilidad concreta: sincronización inicial de datos, autenticación y gestión de usuarios, gestión del perfil y de la visibilidad pública, relaciones de “siguiendo”, administración de categorías y preguntas, obtención de algoritmos y actualización de estadísticas.

También se ha diseñado y desplegado un esquema de base de datos relacional en MySQL que almacena usuarios, preguntas, categorías, respuestas y relaciones entre ellos, garantizando la integridad de la información. Todo el conjunto se ha orquestado mediante Docker y ficheros *docker-compose*, lo que facilita el despliegue unificado de la aplicación y sienta las bases para su escalabilidad y mantenimiento futuro.

Desde el punto de vista funcional, la aplicación cumple los requisitos definidos: permite registrarse e iniciar sesión, seleccionar diferentes algoritmos de estudio, responder sesiones de preguntas, actualizar automáticamente las estadísticas del usuario y consultar su progreso mediante gráficas y paneles de información. Además, incorpora una capa social que posibilita seguir a otros usuarios con perfil público y comparar los resultados propios con los de ellos, reforzando así el componente de gamificación y comparación social planteado en el proyecto.

A modo de conclusión personal, este trabajo me ha permitido afianzar de forma práctica mis conocimientos en desarrollo web y en arquitecturas de microservicios. Considero especialmente positivo haber trabajado con este enfoque, que resulta versátil y escalable, y que me gustaría seguir utilizando en futuros proyectos.

6.2 TRABAJO FUTURO

Este proyecto dista mucho de estar acabado, hay bastantes áreas de mejora que se podrían afrontar en el futuro. En este apartado se indican algunas posibles mejoras que se podrán aplicar más adelante.

Una mejora muy importante y que encaja bien con el proyecto es desplegar esta aplicación en la nube, para poder así ser accedida desde distintos dispositivos. Eso haría que fuese mucho más fácil aprender con esta aplicación, porque no requeriría de preparación previa para usarla.

Otra mejora que podría ser interesante añadir es que la aplicación fuese *responsive*. De momento se puede utilizar en dispositivos como ordenadores, pero que se pudiera usar también en dispositivos móviles haría el aprendizaje bastante más cómodo por cómo está diseñada la aplicación.

BIBLIOGRAFÍA

[1] W. v. d. B. Damien S. Fleur and B. Bredeweg, "Social comparison in learning analytics dashboard supporting motivation and academic achievement," *Computers and Education Open*, vol. 4, December 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666557323000083

[2] W. Tong and G. Shakibaei, "The role of social comparison in online learning motivation through the lens of social comparison theory," *Acta Psychologica*, vol. 258, August 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0001691825006043

[3] A. Joshi, "Supporting student motivation through social comparison," September 2022. [Online]. Available: https://ceur-ws.org/Vol-3292/DCECTEL2022_paper03.pdf

[4] "Herramienta en línea gratuita de desarrollo web | figma," https://www.figma.com/es-es/web-development/, 2025, accedido en 2025.

[5] "Diagrams.net (anteriormente draw.io)," https://www.diagrams.net/, 2025, sitio oficial, consultado en 2025.

[6] P. S. Emmanni, "The role of typescript in enhancing development with modern javascript frameworks," *International Journal of Science and Research (IJSR)*, vol. 10, no. 2, 2021, accedido en 2025. [Online]. Available: https://www.ijsr.net

[7] "Node.js," https://nodejs.org/, 2025, sitio oficial, consultado en 2025.

[8] "Express.js," https://expressjs.com/, 2025, sitio oficial, consultado en 2025.

[9] "Stack overflow developer survey 2025 – technology," https://survey.stackoverflow.co/2025/technology, 2025, accedido en 2025.

[10] "Flask documentation," https://flask.palletsprojects.com/, 2025, accedido en 2025.

[11] "React.js overview," https://react.dev/, 2025, accedido en 2025.

[12] "Chart.js documentation," https://www.chartjs.org/, 2025, accedido en 2025.

[13] "Visual studio code features," https://code.visualstudio.com/, 2025, accedido en 2025.

[14] "Mysql," https://www.mysql.com/, 2026, sitio oficial, consultado en 2026.

[15] "Vite — next generation frontend tooling," https://vite.dev/, 2026, sitio oficial — consultado el 2026-01-03.

[16] “Docker: Accelerated container application development,” https://www.docker.com/, 2026, sitio oficial — consultado el 2026-01-03.

[17] “Install project - microsoft support,” https://support.microsoft.com/en-us/office/install-project-7059249b-d9fe-4d61-ab96-5c5bf435f281, 2026, página de soporte oficial de Microsoft — consultado el 2026-01-03.

[18] “Google workspace: Secure online productivity & collaboration tools,” https://workspace.google.com/, 2026, sitio oficial — consultado el 2026-01-03.

[19] “Overleaf, online latex editor,” https://www.overleaf.com/, 2026, sitio oficial — consultado el 2026-01-03.

[20] A. Variados, “La simulación clínica en la educación médica moderna: revisión de revisiones,” *Revista Eugenio Espejo*, vol. 19, no. 1, pp. 102–116, 2025, simulación clínica mejora competencias y retención de conocimientos.

[21] A. med, “El estetoscopio: desde sus inicios hasta ti,” July 2020. [Online]. Available: https://neurologia.almirallmed.es/blog/el-estetoscopio-desde-sus-inicios-hasta-ti/

[22] A. Variados, “La simulación clínica en la enseñanza de la cardiología: Revisión sistemática,” *Revista Andina de Educación*, vol. 8, no. 1, p. 000813, 2024, revisión sobre simulación para enseñanza de auscultación.

[23] V. M. García Macías and otros, “Aprendizaje adaptativo a través de la inteligencia artificial en la educación superior,” *Revista Científica de Innovación Educativa y Sociedad Actual*, vol. 5, no. 4, pp. 480–489, 2025.

[24] M. F. Yugcha Ulpo, C. L. Montalv0̆0e1n V0̆0e9lez, B. M. Acurio Acurio, S. D. C. Mendoza Mendoza, and K. A. Orna Quintanilla, “Ludificaci0̆0f3n y su influencia en el aprendizaje de los estudiantes en la educaci0̆0f3n actual: un enfoque investigativo,” *Revista Cient0̆0edfica Multidisciplinar G-Nerando*, vol. 4, no. 1, pp. 1–9, 2023. [Online]. Available: https://revista.gnerando.org/revista/index.php/RCMG/article/view/68

[25] W. Luo, M. Zhang, and J. Jia, “Gamified learning and motivation: A systematic review and meta-analysis,” *Computers & Education*, vol. XXX, pp. YYY–ZZZ, 2023, revisión sobre aprendizaje gamificado y efectos motivacionales.

[26] A. Variados, “Prácticas de los equipos de desarrollo de microservicios: un mapeo sistemático de la literatura,” *ReCIBE. Revista electrónica de Computación, Informática, Biomédica y Electrónica*, vol. 10, no. 1, pp. 1–26, 2021, mapeo sistemático sobre prácticas y retos del desarrollo de microservicios.

[27] L. stethoscopes, “3m™ littmann® learning institute app,” Tech. Rep. [Online]. Available: https://www.littmann.in/3M/en_IN/littmann-stethoscopes-in/education/lli-app/

[28] eMurmur, “emurmur,” Tech. Rep. [Online]. Available: https://emurmur.com/

[29] Thinklabs, “Thinklabs imurmur,” Tech. Rep. [Online]. Available: https://www.thinklabs.com/

UNIVERSIDAD DE ALMERÍA



Resumen/Abstract

Este Trabajo de Fin de Grado tiene como propósito desarrollar una aplicación que ayude a estudiantes de medicina a reconocer sonidos corporales a partir de audios reales. Para ello, se ha diseñado una arquitectura basada en microservicios junto con un sistema de recomendación y elementos de gamificación.

El resultado es una aplicación funcional que genera sesiones de práctica personalizadas, permite gestionar preguntas y categorías, y ofrece estadísticas comparativas. Se concluye que la solución es escalable y constituye un recurso útil para apoyar el entrenamiento en auscultación.

The purpose of this Final Degree Project is to develop an application that helps medical students recognize body sounds using real audio samples. The system applies a microservices architecture together with a recommendation mechanism and gamification elements.

The final application is fully functional, generating personalized practice sessions, managing questions and categories, and providing comparative statistics. The results show that the solution is scalable and offers a useful tool to support auscultation training.

GRADO EN INGENIERÍA INFORMÁTICA

2025/2026